

Vietnam–Korea University of Information and Communication Technology

VKU_TLE_WA_CE

Vo Nguyen Hoang An, Nguyen Tung Lam, Nguyen Van Phung

The 2026 ICPC Vietnam National Contest

Nov 08, 2026

1 Contest 1

2 Math 1

3 Data structures 4

4 Numerical 7

5 Number theory 9

6 Combinatorics 12

7 Graph 13

8 Geometry 17

9 Strings 18

10 Various 20

Contest (1)

template.cpp — 16 lines

```cpp
#include <bits/stdc++.h>
using namespace std;
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#define rep(i, a, b) for (int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()
#define all(x) begin(x), end(x)
#define rall(x) rbegin(x), rend(x)
#define nl "\n"
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

template.py — 21 lines

```python
import sys
sys.setrecursionlimit(10000000)
from collections import deque, Counter, defaultdict
from bisect import bisect_left, bisect_right, insort
from heapq import (
    heapify, heappush, heappop,
    heappushpop, heapreplace,
    nlargest, nsmallest
)
from itertools import (
    accumulate, permutations, combinations,
    combinations_with_replacement,
    product, groupby, chain
)
from functools import lru_cache, cmp_to_key, reduce
from operator import itemgetter, mul, xor
from fractions import Fraction
from decimal import Decimal, getcontext
getcontext().prec = 20
from array import array
points.sort(key=lambda p: (p[0], -p[1]))
```

.bashrc — 6 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 -
    fsanitize=undefined,address'
alias co='g++ -O2 -std=c++17'
ulimit -s unlimited
xmodmap -e 'clear lock' -e 'keycode 66 = Control_L' -e 'add
    Control = Control_L'
setxkbmap -option ctrl:nocaps
gsettings set org.gnome.desktop.input-sources xkb-options "['
    ctrl:nocaps']"
```

.vimrc — 5 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on | im jk <esc> | im kj <esc> | no ; :
" Select region and then type :Hash to hash your selection.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
    md5sum \| cut -c-6
nnoremap <F1> :w<CR>:!g++ -Wall -Wconversion -Wfatal-errors -g
    -std=c++17 -fsanitize=undefined,address % -o %:r && ./%:r<
    CR>
```

hash.sh — 3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

troubleshoot.txt — 48 lines

```
- Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.
- Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
- Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).
- Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?
- Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Math (2)

2.1 Phương trình

2.1.1 Phương trình bậc hai

Xét phương trình $ax^2 + bx + c = 0$ ($a \neq 0$), $\Delta = b^2 - 4ac$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Tọa độ đỉnh parabol (cực trị): Đạt tại $x = -\frac{b}{2a}$, giá trị tương ứng $y = -\frac{\Delta}{4a}$

Định lý Viète: $x_1 + x_2 = \frac{b}{a} \quad x_1 x_2 = \frac{c}{a}$

2.1.2 Hệ phương trình bậc nhất hai ẩn (Quy tắc Cramer 2x2)

Dùng để giải hệ phương trình tuyến tính hoặc tìm nhanh giao điểm 2 đường thẳng:

$$\begin{cases} ax + by = e \\ cx + dy = f \end{cases} \implies x = \frac{ed - bf}{ad - bc}, \quad y = \frac{af - ec}{ad - bc}$$

Nếu định thức $D = ad - bc = 0$: Hệ vô nghiệm (hai đường song song) hoặc vô số nghiệm (hai đường trùng nhau).

2.1.3 Hệ phương trình tuyến tính tổng quát

Cho hệ $Ax = b$ với A là ma trận vuông không suy biến ($\det A \neq 0$):

$$x_i = \frac{\det A'_i}{\det A}$$

Trong đó A'_i là ma trận nhận được từ ma trận A sau khi thay thế cột thứ i bằng vector cột b. (Thực tế chỉ dùng nhẩm tay cho hệ 2×2 và 3×3; hệ lớn hơn giải bằng khử Gauss).

2.2 Công thức truy hồi (Recurrences)

2.2.1 Phương trình đặc trưng (Hệ thức truy hồi tuyến tính thuần nhất)

Xét hệ thức truy hồi tuyến tính cấp k:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

Thiết lập phương trình đa thức đặc trưng:

$$P(x) = x^k - c_1 x^{k-1} - c_2 x^{k-2} - \cdots - c_k = 0$$

Trường hợp nghiệm phân biệt: Nếu phương trình có k nghiệm phân biệt $r_1, r_2, \ldots, r_k$, công thức tổng quát là:

$$a_n = d_1 r_1^n + d_2 r_2^n + \cdots + d_k r_k^n$$

(Xác định các hệ số d_i bằng cách thế các giá trị khởi tạo ban đầu $a_0, a_1, \ldots, a_{k-1}$ để giải hệ phương trình tuyến tính)
Trường hợp nghiệm bội: Nếu nghiệm r lặp lại m lần (bội m), nó sẽ đóng góp một nhân tử đa thức bậc $m-1$:

$$(d_1 + d_2 n + d_3 n^2 + \cdots + d_m n^{m-1}) r^n$$

Ví dụ dãy Fibonacci: $F_n = F_{n-1} + F_{n-2}$ với $F_0 = 0, F_1 = 1$.
Phương trình đặc trưng: $x^2 - x - 1 = 0 \implies r = \frac{1 \pm \sqrt{5}}{2}$.
Công thức Binet dạng đóng:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$$

2.2.2 Nhân ma trận giải công thức truy hồi (Matrix Exponentiation)

Áp dụng khi cần tính số hạng thứ n với giá trị n rất lớn ($n \le 10^{18}$) và bậc truy hồi nhỏ ($k \le 50$) trong độ phức tạp $\mathcal{O}(k^3 \log n)$:

$$\begin{bmatrix} a_n \\ a_{n-1} \\ a_{n-2} \\ \vdots \\ a_{n-k+1} \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 & \ldots & c_k \\ 1 & 0 & 0 & \ldots & 0 \\ 0 & 1 & 0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & 0 \end{bmatrix} \begin{bmatrix} a_{n-1} \\ a_{n-2} \\ a_{n-3} \\ \vdots \\ a_{n-k} \end{bmatrix}$$

2.3 Lượng giác

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

với $r = \sqrt{a^2+b^2}$, $\phi = \operatorname{atan2}(b, a)$.

$$\sin(a \pm b) = \sin a \cos b \pm \cos a \sin b$$
$$\cos(a \pm b) = \cos a \cos b \mp \sin a \sin b$$
$$\tan(a \pm b) = \frac{\tan a \pm \tan b}{1 \mp \tan a \tan b}$$
$$\sin 2a = 2\sin a \cos a, \qquad \cos 2a = \cos^2 a - \sin^2 a = 2\cos^2 a - 1$$
$$\sin 3a = 3\sin a - 4\sin^3 a, \qquad \cos 3a = 4\cos^3 a - 3\cos a$$
$$\sin a \pm \sin b = 2\sin\frac{a \pm b}{2}\cos\frac{a \mp b}{2}$$
$$\cos a + \cos b = 2\cos\frac{a+b}{2}\cos\frac{a-b}{2}$$
$$\cos a - \cos b = -2\sin\frac{a+b}{2}\sin\frac{a-b}{2}$$

Đặt $t = \tan\frac{x}{2}$: $\sin x = \frac{2t}{1+t^2}$, $\cos x = \frac{1-t^2}{1+t^2}$.

2.4 Hình học

2.4.1 Tam giác

Cho tam giác có độ dài ba cạnh a, b, c. **Nửa chu vi:**

$$p = \frac{a+b+c}{2}$$

Diện tích (công thức Heron):

$$A = \sqrt{p(p-a)(p-b)(p-c)}$$

Bán kính đường tròn ngoại tiếp:

$$R = \frac{abc}{4A}$$

Bán kính đường tròn nội tiếp:

$$r = \frac{A}{p}$$

Độ dài đường trung tuyến: Đường trung tuyến chia tam giác thành hai tam giác có diện tích bằng nhau.

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Độ dài đường phân giác: Đường phân giác chia một góc thành hai góc bằng nhau.

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Định lý sin:

$$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$$

Định lý cosin:

$$a^2 = b^2 + c^2 - 2bc\cos\alpha$$

Định lý tang:

$$\frac{a+b}{a-b} = \frac{\tan\left(\frac{\alpha+\beta}{2}\right)}{\tan\left(\frac{\alpha-\beta}{2}\right)}$$

2.4.2 Tứ giác

Cho tứ giác có độ dài các cạnh a, b, c, d, hai đường chéo e, f, góc giữa hai đường chéo θ, diện tích A và đặt

$$F = b^2 + d^2 - a^2 - c^2.$$

Ta có:

$$4A = 2ef\sin\theta = F\tan\theta$$
$$= \sqrt{4e^2f^2 - F^2}.$$

Đối với tứ giác nội tiếp, tổng hai góc đối bằng $180°$. Ngoài ra, ta có:

$$ef = ac + bd,$$
$$A = \sqrt{(p-a)(p-b)(p-c)(p-d)},$$

trong đó $p = \frac{a+b+c+d}{2}$ là nửa chu vi tứ giác.

2.4.3 Hình học 3D

Với $a = (a_x, a_y, a_z)$, $b = (b_x, b_y, b_z)$:

$$a \times b = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$$
$$|a \times b| = |a||b|\sin\theta, \quad a \cdot b = |a||b|\cos\theta$$

$a \times b$ vuông góc với cả a và b (quy tắc bàn tay phải), $b \times a = -a \times b$.

- Tích hỗn hợp: $[a, b, c] = a \cdot (b \times c) = \det[a\ b\ c]$. Dấu cho biết hướng a, b, c.
- Diện tích tam giác ABC: $\frac{1}{2}|(B-A) \times (C-A)|$.
- Thể tích hình hộp: $|[a, b, c]|$. Thể tích tứ diện $ABCD$: $\frac{1}{6}\left|(B-A) \cdot \big((C-A) \times (D-A)\big)\right|$.
- Mặt phẳng qua A, B, C: pháp vector $n = (B-A) \times (C-A)$, phương trình $n \cdot (X-A) = 0$, tức $ax + by + cz + d = 0$ với $(a, b, c) = n$, $d = -n \cdot A$.
- Điểm P tới mặt phẳng: $\frac{|n \cdot (P-A)|}{|n|}$.
- Điểm P tới đường thẳng qua A, hướng d: $\frac{|(P-A) \times d|}{|d|}$.
- Giao đường thẳng $X = P + td$ với mặt phẳng (n, A): $t = \frac{n \cdot (A-P)}{n \cdot d}$ (nếu $n \cdot d = 0$ thì song song hoặc nằm trong mặt phẳng).
- Khoảng cách hai đường chéo nhau (qua A_1 hướng d_1, qua A_2 hướng d_2): $\frac{|(A_2 - A_1) \cdot (d_1 \times d_2)|}{|d_1 \times d_2|}$.
- Góc giữa hai mặt phẳng: $\cos\varphi = \frac{|n_1 \cdot n_2|}{|n_1||n_2|}$.
- Quay v quanh trục đơn vị k một góc θ (Rodrigues): $v' = v\cos\theta + (k \times v)\sin\theta + k(k \cdot v)(1 - \cos\theta)$.
- Cầu: $V = \frac{4}{3}\pi r^3$, $S = 4\pi r^2$. Nón: $V = \frac{1}{3}\pi r^2 h$. Chỏm cầu cao h: $V = \frac{\pi h^2(3r-h)}{3}$.

2.5 Tổng

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, \quad c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.5.1 Chia kẹo (Stars and Bars)

Số nghiệm nguyên của $x_1 + x_2 + \cdots + x_k = n$ (chia n vật giống nhau vào k hộp phân biệt): $x_i \geq 0$: $\binom{n+k-1}{k-1}$

$x_i \geq 1$ (cần $n \geq k$): $\binom{n-1}{k-1}$

$x_i \geq c_i$: đặt $n' = n - \sum c_i$, nếu $n' \geq 0$ thì đáp án là $\binom{n'+k-1}{k-1}$, ngược lại bằng 0

$x_i \leq u_i$: bù trừ theo tập S các biến vi phạm ($x_i \geq u_i + 1$): $\sum_{S \subseteq [k]} (-1)^{|S|} \binom{n - \sum_{i \in S}(u_i+1)+k-1}{k-1}$

bỏ các số hạng có $n - \sum_{i \in S}(u_i + 1) < 0$. Nếu mọi $u_i = u$ thì $\sum_{j=0}^{k} (-1)^j \binom{k}{j} \binom{n-j(u+1)+k-1}{k-1}$

2.6 Hệ số nhị thức và đa thức

2.6.1 Công thức cơ bản

$$\binom{n}{r} = \frac{n!}{r!(n-r)!} = \binom{n}{n-r}, \binom{n}{r} = 0 \quad (r < 0 \text{ hoặc } r > n)$$

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r} = \binom{n}{r-1} \frac{n-r+1}{r}.$$

$$\sum_r \binom{n}{r} = 2^n, \quad \sum_r \binom{n}{r}^2 = \binom{2n}{n}.$$

$$\sum_{i=0}^{r} \binom{m}{i} \binom{n}{r-i} = \binom{m+n}{r}, \sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

$\binom{n}{r}$ lẻ $\iff$ `(n & r) == r`.

2.6.2 Tính $\binom{n}{r}$

BinomExact.h

Description: hàm này để tính chính xác $\binom{n}{r}$ (không lấy modulo) bằng cách nhân chia xen kẽ. Sau bước i, `res` $= \binom{n}{i+1}$ nên phép chia luôn chia hết. Kết quả phải vừa trong `ll`; tích trung gian dùng `__int128` để không bị tràn. Trả về 0 nếu $r < 0$ hoặc $r > n$.

Time: $\mathcal{O}(\min(r, n-r))$

14a3ef, 9 lines

```
ll Cexact(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    r = min(r, n - r);
    ll res = 1;
    for (ll i = 0; i < r; i++)
        res = (__int128)res * (n - i) / (i + 1);
    return res;
}
```

BinomMod.h

Description: hàm này để tính $\binom{n}{r}$ mod p với p nguyên tố và $n \leq \min(N-1, p-1)$. `init()` tiền xử lý giai thừa và nghịch đảo giai thừa (đặt `MOD` trước khi gọi), sau đó mỗi truy vấn `C(n, r)` mất $\mathcal{O}(1)$. Nếu $n \geq p$ thì dùng `lucas` trong Lucas.h.

Usage: `MOD = 1000000007; init(); C(10, 3);`

Time: $\mathcal{O}(N)$, mỗi truy vấn $\mathcal{O}(1)$

bc7d44, 24 lines

```
const int N = 1000005;
ll MOD = 1000000007, fact[N], ifact[N];
ll pw(ll a, ll e) {
    ll r = 1;
    a %= MOD;
    for (; e; e >>= 1, a = a * a % MOD)
        if (e & 1)
            r = r * a % MOD;
    return r;
}
void init() {
    int L = min<ll>(N - 1, MOD - 1);
    fact[0] = 1;
    for (int i = 1; i <= L; i++)
        fact[i] = fact[i - 1] * i % MOD;
    ifact[L] = pw(fact[L], MOD - 2);
    for (int i = L; i; i--)
        ifact[i - 1] = ifact[i] * i % MOD;
}
ll C(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    return fact[n] * ifact[r] % MOD * ifact[n - r] % MOD;
}
```

2.6.3 Định lý Lucas

Cho p là số nguyên tố. Viết biểu diễn cơ số p của n và r:

$$n = \sum_{i=0}^{k} n_i p^i, \qquad r = \sum_{i=0}^{k} r_i p^i.$$

Theo định lý Lucas:

$$\binom{n}{r} \equiv \prod_{i=0}^{k} \binom{n_i}{r_i} \pmod{p}.$$

Kết quả bằng 0 nếu tồn tại i sao cho $n_i < r_i$.

Lucas.h

Description: hàm này để tính $\binom{n}{r}$ mod p với p là số nguyên tố nhỏ ($p \leq N$) còn n, r rất lớn (tới 10^{18}). Theo định lý Lucas, $\binom{n}{r} \equiv \prod_i \binom{n_i}{r_i} \pmod{p}$ với n_i, r_i là các chữ số của n, r trong hệ cơ số p (bằng 0 nếu có $n_i < r_i$). Cần BinomMod.h: đặt `MOD` $= p$ rồi gọi `init()` trước.

Usage: `MOD = 99991; init(); lucas(1e18, 12345);`

Time: $\mathcal{O}(p + \log_p n)$

d48bc5, 8 lines

```
ll lucas(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    ll res = 1;
    for (; n || r; n /= MOD, r /= MOD)
        res = res * C(n % MOD, r % MOD) % MOD;
    return res;
}
```

2.6.4 Tổng $\sum_{k=0}^{n} P(k)\binom{n}{k}$ với n lớn, P bậc nhỏ

Đặt giai thừa giảm:

$$k^{\underline{r}} = k(k-1)\cdots(k-r+1).$$

Ta có đồng nhất thức:

$$k^{\underline{r}} \binom{n}{k} = n^{\underline{r}} \binom{n-r}{k-r}.$$

Suy ra:

$$\sum_{k=0}^{n} k^{\underline{r}} \binom{n}{k} = n^{\underline{r}} \sum_{k=r}^{n} \binom{n-r}{k-r} = n^{\underline{r}} 2^{n-r}.$$

Quy ước kết quả bằng 0 nếu $r > n$.

Một số trường hợp thường gặp:

$$\sum_{k=0}^{n} k \binom{n}{k} = n2^{n-1}$$

$$\sum_{k=0}^{n} k^2 \binom{n}{k} = n(n+1)2^{n-2}.$$

SumPoly.h

Description: hàm này để tính $\sum_{k=0}^{n} P(k)\binom{n}{k}$ theo modulo MOD, với n rất lớn (tới 10^{18}) và P có bậc nhỏ. Viết $P(k) = \sum_r c_r (k)_r$ với $(k)_r = k(k-1)\cdots(k-r+1)$ là giai thừa giảm, khi đó tổng bằng $\sum_r c_r (n)_r 2^{n-r}$. Đổi cơ sở: $k^2 = (k)_2 + (k)_1$, $k^3 = (k)_3 + 3(k)_2 + (k)_1$, $k^4 = (k)_4 + 6(k)_3 + 7(k)_2 + (k)_1$. Cần BinomMod.h cho pw và MOD.

Usage: `sumPoly(n, {0, 1, 1}); // tong k*k*C(n,k)`

Time: $\mathcal{O}(d \log n)$ với d là bậc của P

1a9318, 8 lines

```
ll sumPoly(ll n, const vector<ll> &c) {
    ll res = 0, f = 1; // f = (n)_r mod MOD
    for (int r = 0; r < (int)c.size() && r <= n; r++) {
        res = (res + c[r] % MOD * f % MOD * pw(2, n - r)) % MOD
            ;
        f = f * ((n - r) % MOD) % MOD;
    }
    return res;
}
```

2.6.5 Hệ số đa thức và xâu thứ k theo từ điển

Số hoán vị của n phần tử, loại i có a_i phần tử: $\binom{n}{a_1, \ldots, a_m} = \frac{n!}{a_1! \cdots a_m!} = \binom{n}{a_1}\binom{n-a_1}{a_2} \cdots$.

KthPermutation.h

Description: hàm này để tìm xâu (hoán vị có lặp) thứ k theo thứ tự từ điển, k đánh số từ 1. `a[i]` là số lần xuất hiện của ký tự `'a'+i`. Trả về xâu rỗng nếu k lớn hơn tổng số hoán vị. Thuật toán tham lam: ở mỗi vị trí thử ký tự c tăng dần, `cnt` là số hoán vị của phần còn lại $\binom{n-1}{a_1, \ldots, a_c - 1, \ldots, a_m}$; nếu $k \leq$ `cnt` thì chốt c, ngược lại $k -=$ `cnt`. Mọi phép cộng/nhân bị chặn trên bởi `INF` để không tràn số (không cần BigInt). Gọi `initPascal()` một lần ở đầu.

Usage: `initPascal(); kth({2, 1, 1}, 5); // các ký tự a,a,b,c: aabc, aacb, abac, abca, acab => "acab"`

Time: $\mathcal{O}(M^2)$ khởi tạo, $\mathcal{O}(n \cdot m^2)$ mỗi truy vấn

d1e070, 38 lines

```
const ll INF = 2e18;
const int M = 2005; // độ dài xâu tối đa
ll Cs[M][M];
void initPascal() {
    for (int i = 0; i < M; i++) {
        Cs[i][0] = 1;
        for (int j = 1; j <= i; j++)
            Cs[i][j] = min(INF, Cs[i - 1][j - 1] + Cs[i - 1][j
                ]);
    }
}
ll mulSat(ll a, ll b) { return (a && b > INF / a) ? INF : a * b
    ; }
ll multinomial(const vector<int> &a, int n) {
    ll res = 1;
    for (int x : a) {
        res = mulSat(res, Cs[n][x]);
        n -= x;
    }
    return res;
}
string kth(vector<int> a, ll k) {
    int n = accumulate(a.begin(), a.end(), 0);
    if (multinomial(a, n) < k)
        return "";
    string s;
    for (int pos = 0; pos < n; pos++)
        for (int c = 0; c < (int)a.size(); c++)
            if (a[c]) {
                a[c]--;
                ll cnt = multinomial(a, n - pos - 1);
                if (k <= cnt) {
                    s += char('a' + c);
                    break;
                }
                k -= cnt;
                a[c]++;
            }
    return s;
}
```

2.7 Lý thuyết xác suất

Gọi X là biến ngẫu nhiên rời rạc với xác suất nhận giá trị x là $p_X(x)$. Khi đó, giá trị kỳ vọng (trung bình) của nó là $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ và phương sai là $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$, trong đó σ là độ lệch chuẩn. Nếu X là biến ngẫu nhiên liên tục, nó sẽ có hàm mật độ xác suất $f_X(x)$, và các tổng ở trên được thay bằng tích phân với $p_X(x)$ thay bằng $f_X(x)$

Kỳ vọng có tính tuyến tính:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

Đối với X và Y độc lập:

$$V(aX + bY) = a^2V(X) + b^2V(Y)$$

2.7.1 Các phân phối rời rạc

Phân phối nhị thức (Binomial distribution)

Số lần thành công trong n phép thử độc lập có kết quả có/không (thành công/thất bại), mỗi phép thử có xác suất thành công là p, tuân theo phân phối $\text{Bin}(n, p)$, $n = 1, 2, \ldots$, $0 \le p \le 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\text{Bin}(n, p)$ xấp xỉ $\text{Po}(np)$ (phân phối Poisson) khi p nhỏ.

Phân phối số lần thử tới thành công đầu tiên (First success distribution)

Số phép thử cần thiết để nhận được lần thành công đầu tiên trong các phép thử độc lập có kết quả có/không, mỗi phép thử có xác suất thành công là p, tuân theo phân phối $\text{Fs}(p)$, $0 \le p \le 1$ (dạng hình học dịch chuyển/Geometric distribution bắt đầu từ 1).

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

2.8 Công thức nhanh

2.8.1 Số học

- Số ước $\tau(n) = \prod(e_i + 1)$, tổng ước $\sigma(n) = \prod \frac{p_i^{e_i+1}-1}{p_i-1}$.
- Wilson: p nguyên tố $\iff (p-1)! \equiv -1 \pmod p$.
- Euler mở rộng (tháp lũy thừa): nếu $n \ge \log_2 m$ thì $a^n \equiv a^{(n \bmod \varphi(m)) + \varphi(m)} \pmod m$.
- Nghịch đảo $1..n$ modulo nguyên tố p: $\text{inv}[i] = -\lfloor p/i \rfloor \cdot \text{inv}[p \bmod i] \bmod p$.
- a là thặng dư bậc hai mod p $\iff a^{(p-1)/2} \equiv 1$ (tiêu chuẩn Euler).

2.8.2 Fibonacci

- $\gcd(F_m, F_n) = F_{\gcd(m,n)}$; $F_{m+n} = F_m F_{n+1} + F_{m-1} F_n$.
- Cassini: $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$.
- Nhân đôi nhanh: $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_k^2 + F_{k+1}^2$.
- $\sum_{i=1}^n F_i = F_{n+2} - 1$, $\sum_{i=1}^n F_i^2 = F_n F_{n+1}$.
- Chu kỳ Pisano: $\pi(2) = 3,\ \pi(5) = 20,\ \pi(10) = 60,\ \pi(10^k) = 15 \cdot 10^{k-1}$ $(k \ge 3)$.

2.8.3 Tổ hợp

- Ballot: số dãy a bước lên, b bước xuống $(a \ge b)$ không bao giờ xuống dưới 0: $\binom{a+b}{b} - \binom{a+b}{b-1}$.
- Vòng/chuỗi tô m màu, quay (Burnside): $\frac{1}{n}\sum_{d|n} \varphi(n/d)\, m^d$.
- Nội suy Lagrange: $P(x) = \sum_i y_i \prod_{j \ne i} \frac{x - x_j}{x_i - x_j}$. $\sum_{i=1}^n i^k$ là đa thức bậc $k+1$ theo $n \Rightarrow$ cần $k+2$ điểm.
- Số đồ thị gán nhãn trên n đỉnh: $2^{\binom{n}{2}}$.

2.8.4 Đồ thị

- Euler (đồ thị phẳng, C thành phần liên thông): $V - E + F = 1 + C$ (liên thông: $= 2$). Đơn đồ thị phẳng, $V \ge 3$: $E \le 3V - 6$.
- Cayley: n^{n-2} cây gán nhãn. Kirchhoff: số cây khung = định thức của ma trận Laplace $L = D - A$ sau khi bỏ một hàng và một cột bất kỳ.
- Đồ thị hai phía: König $\Rightarrow$ min vertex cover = max matching, max independent set = $V-$ max matching.
- Hall: có ghép đầy đủ phía trái $\iff |N(S)| \ge |S|$ với mọi S phía trái.
- Dilworth: số chuỗi ít nhất phủ poset = kích thước phản xích lớn nhất.

2.8.5 Hình học

- Pick (đa giác nguyên): $A = I + \frac{B}{2} - 1$; số điểm nguyên trên đoạn $= \gcd(|\Delta x|, |\Delta y|)$.
- Thể tích tứ diện $ABCD$: $\frac{1}{6}\left|(B - A) \cdot \big((C - A) \times (D - A)\big)\right|$.
- Khoảng cách điểm tới mặt phẳng $ax + by + cz + d = 0$: $\frac{|ax_0 + by_0 + cz_0 + d|}{\sqrt{a^2 + b^2 + c^2}}$.
- Cầu: $V = \frac{4}{3}\pi r^3$, $S = 4\pi r^2$. Nón: $V = \frac{1}{3}\pi r^2 h$. Hình quạt góc θ: $\frac{1}{2}r^2\theta$.

Data structures (3)

OrderStatisticTree.h

Description: Ordered Set (không hỗ trợ phần tử trùng lặp), cho phép tìm phần tử đứng thứ k và xác định thứ hạng của một phần tử. Muốn tạo map, thay `null_type` bằng kiểu dữ liệu tương ứng.

Time: $\mathcal{O}(logN)$ 782797, 16 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template <class T>
using Tree =
  tree<T, null_type, less<T>, rb_tree_tag,
       tree_order_statistics_node_update>;
void example() {
```

```
  Tree<int> t, t2;
  t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map có API gần giống unordered_map nhưng thường nhanh hơn khoảng 3 lần, đổi lại sử dụng nhiều bộ nhớ hơn khoảng 1.5 lần. Dung lượng ban đầu (nếu chỉ định) phải là lũy thừa của 2.

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x * C);
      }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1
    << 16});
```

SegmentTree.h

Description: Segment Tree iterative, đánh chỉ số từ 0. Đoạn truy vấn có dạng [l, r), bao gồm l và không bao gồm r. Mặc định hỗ trợ truy vấn giá trị lớn nhất; có thể thay đổi bằng cách chỉnh T, f và unit.
Time: $\mathcal{O}(logN)$

0f4bdb, 22 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s;
  int n;
  Tree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2)
        ra = f(ra, s[b++]);
      if (e % 2)
        rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegmentTree.h

Description: Segment Tree động hỗ trợ cộng hoặc gán một giá trị trên một đoạn, đồng thời truy vấn giá trị lớn nhất trên đoạn. Các node chỉ được tạo khi cần thiết, phù hợp với miền chỉ số rất lớn. Có thể thay đổi để hỗ trợ các phép toán khác. Nên sử dụng cùng Bump Allocator để tăng tốc độ cấp phát bộ nhớ, và SmallPtr hoặc chỉ số ngầm định để tiết kiệm bộ nhớ.
Usage: `Node* tr = new Node(v, 0, sz(v));`
Time: $\mathcal{O}(\log N)$ cho mỗi thao tác.

"../various/BumpAllocator.h" 34ecf5, 58 lines

```
const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf;
  Node(int lo, int hi) : lo(lo), hi(hi) {} // Large interval of
      -inf
  Node(vi &v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo + 1 < hi) {
      int mid = lo + (hi - lo) / 2;
      l = new Node(v, lo, mid);
      r = new Node(v, mid, hi);
      val = max(l->val, r->val);
    } else
      val = v[lo];
  }
  int query(int L, int R) {
    if (R <= lo || hi <= L)
      return -inf;
    if (L <= lo && hi <= R)
      return val;
    push();
    return max(l->query(L, R), r->query(L, R));
  }
  void set(int L, int R, int x) {
    if (R <= lo || hi <= L)
      return;
    if (L <= lo && hi <= R)
      mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void add(int L, int R, int x) {
    if (R <= lo || hi <= L)
      return;
    if (L <= lo && hi <= R) {
      if (mset != inf)
        mset += x;
      else
        madd += x;
      val += x;
    } else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void push() {
    if (!l) {
      int mid = lo + (hi - lo) / 2;
      l = new Node(lo, mid);
      r = new Node(mid, hi);
    }
    if (mset != inf)
      l->set(lo, hi, mset), r->set(lo, hi, mset), mset = inf;
    else if (madd)
      l->add(lo, hi, madd), r->add(lo, hi, madd), madd = 0;
  }
};
```

UnionFindRollback.h

Description: Cấu trúc dữ liệu Disjoint Set Union (DSU) hỗ trợ hoàn tác. Nếu không cần hoàn tác, bỏ st, time() và rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log N)$

de4ad0, 25 lines

```
struct RollbackUF {
  vi e;
  vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i-- > t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b)
      return false;
    if (e[a] > e[b])
      swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b];
    e[b] = a;
    return true;
  }
};
```

SubMatrix.h

Description: Tính nhanh tổng các phần tử trong ma trận con, xác định bởi góc trên bên trái và góc dưới bên phải (đoạn nửa kín).
Usage: `SubMatrix<int> m(matrix);`
`m.sum(0, 0, 2, 2); // tổng 4 phần tử ở góc trên bên trái`
Time: $\mathcal{O}\left(N^2 + Q\right)$

c59ada, 12 lines

```
template <class T> struct SubMatrix {
  vector<vector<T>> p;
  SubMatrix(vector<vector<T>> &v) {
    int R = sz(v), C = sz(v[0]);
    p.assign(R + 1, vector<T>(C + 1));
    rep(r, 0, R) rep(c, 0, C) p[r + 1][c + 1] =
      v[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
  }
  T sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
  }
};
```

Matrix.h

Description: Các phép toán cơ bản trên ma trận vuông.
Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

<cassert> 4da5a2, 26 lines

```
template <class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M &m) const {
    M a;
    rep(i, 0, N) rep(j, 0, N) rep(k, 0, N) a.d[i][k] += d[i][j]
        * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N> &vec) const {
    array<T, N> ret{};
    rep(i, 0, N) rep(j, 0, N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i, 0, N) a.d[i][i] = 1;
    while (p) {
      if (p & 1)
        a = a * b;
      b = b * b;
      p >>= 1;
```

```
    }
    return a;
  }
};
```

LineContainer.h

Description: Cấu trúc dữ liệu lưu trữ các đường thẳng dạng kx + m, hỗ trợ thêm đường thẳng và truy vấn giá trị lớn nhất tại một điểm x. Hữu ích trong quy hoạch động (Convex Hull Trick).
Time: $\mathcal{O}(\log N)$

<cassert> 8ec1c7, 36 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line &o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end())
      return x->p = inf, 0;
    if (x->k == y->k)
      x->p = x->m > y->m ? inf : -inf;
    else
      x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z))
      z = erase(z);
    if (x != begin() && isect(--x, y))
      isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

FenwickTree.h

Description: Tính tổng tiền tố a[0] + a[1] + ... + a[pos - 1] và hỗ trợ cập nhật một phần tử a[i] bằng cách cộng độ chênh lệch giữa giá trị mới và giá trị cũ vào phần tử đó.
Time: Both operations are $\mathcal{O}(\log N)$.

e62fac, 25 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1)
      s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1)
      res += s[pos - 1];
    return res;
  }
  int lower_bound(ll sum) { // min pos st sum of [0, pos] >=
      sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0)
      return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
        pos += pw, sum -= s[pos - 1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Tính tổng tiền tố các phần tử a[i][j] với i < I, j < J và hỗ trợ cập nhật giá trị của một phần tử a[i][j]. Cần biết trước các vị trí sẽ được cập nhật: gọi fakeUpdate() cho tất cả các vị trí đó trước khi gọi init().
Time: $\mathcal{O}\left(\log^2 N\right)$. (Dùng Persistent Segment Tree để đạt $\mathcal{O}(\log N)$.)

"FenwickTree.h" 157f07, 26 lines

```
struct FT2 {
  vector<vi> ys;
  vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1)
      ys[x].push_back(y);
  }
  void init() {
    for (vi &v : ys)
      sort(all(v)), ft.emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin());
  }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x - 1].query(ind(x - 1, y));
    return sum;
  }
};
```

RMQ.h

Description: Truy vấn giá trị nhỏ nhất trên một đoạn của mảng. Trả về min(V[a], V[a + 1], ..., V[b - 1]) trong thời gian O(1).
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

<cassert> 510c32, 15 lines

```
template <class T> struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T> &V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j, 0, sz(jmp[k])) jmp[k][j] =
        min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

BinaryTrie.h

Description: Binary Trie (0-1 Trie) hỗ trợ thêm, xóa phần tử và truy vấn XOR trên các số nguyên không âm, bao gồm XOR lớn nhất, nhỏ nhất và đếm số phần tử y thỏa mãn $(x^y) <= k$
Usage:
```
- BinaryTrie tr;
- tr.insert(x); tr.erase(x);
- tr.maxXor(x); // trả về max (x ^ y) với y thuộc Trie
- tr.minXor(x); // trả về min (x ^ y) với y thuộc Trie
- tr.countLeq(x, k); // đếm số y trong Trie thỏa mãn (x ^ y) <= k
```
Time: $\mathcal{O}(B)$ cho mỗi thao tác, với B là số bit của giá trị (B = 30 nếu x <= 1e9).

<cassert> 35c8cd, 72 lines

```
struct BinaryTrie {
  static const int B = 30; // 30 bits for values < 2^30 (~1e9)
  struct Node {
    int ch[2] = {0, 0};
    int cnt = 0;
  };
  vector<Node> t = {Node()};
  void insert(int x, int v = 1) { // v = -1 to erase
    int u = 0;
    t[u].cnt += v;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (!t[u].ch[b]) {
        t[u].ch[b] = sz(t);
        t.emplace_back();
      }
      u = t[u].ch[b];
      t[u].cnt += v;
    }
  }
  void erase(int x) { insert(x, -1); }
  int maxXor(int x) {
    assert(t[0].cnt > 0);
    int u = 0, res = 0;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (t[u].ch[!b] && t[t[u].ch[!b]].cnt > 0) {
        res |= (1 << i);
        u = t[u].ch[!b];
      } else {
        u = t[u].ch[b];
      }
    }
    return res;
  }
  int minXor(int x) {
    assert(t[0].cnt > 0);
    int u = 0, res = 0;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (t[u].ch[b] && t[t[u].ch[b]].cnt > 0) {
        u = t[u].ch[b];
      } else {
        res |= (1 << i);
        u = t[u].ch[!b];
      }
    }
    return res;
  }
  // Count number of elements y in trie such that (x ^ y) <= k
  int countLeq(int x, int k) {
    if (k < 0)
      return 0;
    if (k >= (1 << B))
      return t[0].cnt;
```

```
    int u = 0, ans = 0;
    for (int i = B - 1; i >= 0; --i) {
      int bx = (x >> i) & 1, bk = (k >> i) & 1;
      if (bk) {
        if (t[u].ch[bx])
          ans += t[t[u].ch[bx]].cnt;
        u = t[u].ch[!bx];
      } else {
        u = t[u].ch[bx];
      }
      if (!u)
        return ans;
    }
    return ans + t[u].cnt;
  }
};
```

MoQueries.h

Description: Xử lý các truy vấn trên đoạn hoặc đường đi trong cây bằng cách sắp xếp truy vấn theo thứ tự gần với một hành trình TSP, sau đó chuyển từ truy vấn này sang truy vấn tiếp theo bằng cách thêm hoặc xóa các phần tử ở hai đầu đoạn. Nếu giá trị nằm trên các cạnh của cây, sửa step để thêm/xóa cạnh (a, c) và bỏ lời gọi add ban đầu (nhưng giữ lại in).
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$ 1af40b, 70 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc(){...}        // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first / blk, x.second ^ -(x.first / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t) { return K(Q[s]) < K(Q[t]); })
      ;
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first)
      add(--L, 0);
    while (R < q.second)
      add(R++, 1);
    while (L < q.first)
      del(L++, 0);
    while (R > q.second)
      del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi> &ed, int root =
    0) {
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto &f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep)
      I[x] = N++;
    for (int y : ed[x])
      if (y != p)
        f(y, x, !dep, f);
    if (!dep)
      I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t) { return K(Q[s]) < K(Q[t]); })
      ;
  for (int qi : s)
    rep(end, 0, 2) {
      int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c)

     \
    {

       \
      if (in[c]) {

        del(a, end);
        in[a] = 0;
      } else {

         \
        add(c, end);
        in[c] = 1;
      }

         \
      a = c;

         \
    }
      while (!(L[b] <= L[a] && R[a] <= R[b]))
        I[i++] = b, b = par[b];
      while (a != b)
        step(par[a]);
      while (i--)
        step(I[i]);
      if (end)
        res[qi] = calc();
    }
  return res;
}
```

Numerical (4)

4.1 Optimization

GoldenSectionSearch.h

Description: Tìm vị trí cực tiểu của hàm f trên đoạn [a, b], với giả thiết f là hàm đơn cực (unimodal) trên đoạn này, tức là chỉ có một điểm cực tiểu địa phương và không có điểm cực đại địa phương. Sai số tối đa của kết quả là eps. Có thể sử dụng tương tự để tìm cực đại bằng cách thay đổi một điều kiện nhỏ trong code. Xem TernarySearch.h trong chương Various để biết phiên bản tìm kiếm trên miền rời rạc.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}\left(log((b-a)/eps)\right)$ 31d45b, 20 lines

```
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5) - 1) / 2, eps = 1e-7;
  double x1 = b - r * (b - a), x2 = a + r * (b - a);
  double f1 = f(x1), f2 = f(x2);
  while (b - a > eps)
    if (f1 < f2) { // đổi thành > nếu muốn tìm cực đại
      b = x2;
      x2 = x1;
      f2 = f1;
      x1 = b - r * (b - a);
      f1 = f(x1);
    } else {
      a = x1;
      x1 = x2;
      f1 = f2;
      x2 = a + r * (b - a);
      f2 = f(x2);
    }
  return a;
}
```

4.2 Matrices

IntDeterminant.h

Description: Tính định thức của ma trận bằng các phép toán số học modulo. Có thể bỏ phép lấy modulo để thu được phiên bản tính định thức trên số nguyên thông thường.
Time: $\mathcal{O}\left(N^3\right)$ 3313dc, 20 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>> &a) {
  int n = sz(a);
  ll ans = 1;
  rep(i, 0, n) {
    rep(j, i + 1, n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t)
          rep(k, i, n) a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans)
      return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Giải hệ phương trình tuyến tính A * x = b bằng phương pháp khử Gauss. Nếu hệ có nhiều nghiệm, trả về một nghiệm bất kỳ. Trả về hạng (rank) của ma trận A, hoặc -1 nếu hệ vô nghiệm. Nếu rank < m (số ẩn), hệ có vô số nghiệm. Dữ liệu trong A và b sẽ bị thay đổi trong quá trình thực hiện.
Time: $\mathcal{O}\left(n^2 * m\right)$
<cassert> 44c9ab, 36 lines

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd> &A, vd &b, vd &x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n)
    assert(sz(A[0]) == m);
  vi col(m);
  iota(all(col), 0);
  rep(i, 0, n) {
    double v, bv = 0;
    rep(r, i, n) rep(c, i, m) if ((v = fabs(A[r][c])) > bv) br
        = r, bc = c,
                                bv = v;
    if (bv <= eps) {
      rep(j, i, n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j, 0, n) swap(A[j][i], A[j][bc]);
    bv = 1 / A[i][i];
```

```
    rep(j, i + 1, n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k, i + 1, m) A[j][k] -= fac * A[i][k];
    }
    rank++;
  }
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j, 0, i) b[j] -= A[j][i] * b[i];
  }
  return rank; // rank < m nếu hệ có vô số nghiệm
}
```

SolveLinearBinary.h

Description: Giải hệ phương trình $Ax = b$ trên $\mathbb{F}_2$. Nếu có nhiều nghiệm, trả về một nghiệm bất kỳ. Trả về hạng của ma trận, hoặc -1 nếu hệ vô nghiệm. Phá hủy dữ liệu trong A và b.
Time: $\mathcal{O}\left(n^2 m\right)$

<cassert> fa2d7a, 39 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs> &A, vi &b, bs &x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m);
  iota(all(col), 0);
  rep(i, 0, n) {
    for (br = i; br < n; ++br)
      if (A[br].any())
        break;
    if (br == n) {
      rep(j, i, n) if (b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i - 1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j, 0, n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i);
      A[j].flip(bc);
    }
    rep(j, i + 1, n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i])
      continue;
    x[col[i]] = 1;
    rep(j, 0, i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Tính ma trận nghịch đảo của A. Trả về hạng của ma trận; kết quả được lưu trực tiếp vào A, trừ trường hợp ma trận suy biến (rank < n). Có thể dễ dàng mở rộng cho modulo số nguyên tố. Với modulo lũy thừa của số nguyên tố, lặp lại phép cập nhật: $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ trong đó A^{-1} ban đầu là ma trận nghịch đảo của A modulo p, và k được nhân đôi sau mỗi bước.
Time: $\mathcal{O}\left(n^3\right)$

ebfff6, 37 lines

```
int matInv(vector<vector<double>> &A) {
  int n = sz(A);
  vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i, 0, n) tmp[i][i] = 1, col[i] = i;

  rep(i, 0, n) {
    int r = i, c = i;
    rep(j, i, n) rep(k, i, n) if (fabs(A[j][k]) > fabs(A[r][c])
        ) r = j,
                                  c = k;
    if (fabs(A[r][c]) < 1e-12)
      return i;
    A[i].swap(A[r]);
    tmp[i].swap(tmp[r]);
    rep(j, 0, n) swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j
        ][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j, i + 1, n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k, i + 1, n) A[j][k] -= f * A[i][k];
      rep(k, 0, n) tmp[j][k] -= f * tmp[i][k];
    }
    rep(j, i + 1, n) A[i][j] /= v;
    rep(j, 0, n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n - 1; i > 0; --i)
    rep(j, 0, i) {
      double v = A[j][i];
      rep(k, 0, n) tmp[j][k] -= v * tmp[i][k];
    }

  rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

4.3 Fourier transforms

FastFourierTransform.h

Description: fft(a) tính $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ với mọi k. N phải là lũy thừa của 2. Ứng dụng hữu ích: tính tích chập (convolution): conv(a, b) = c, trong đó $c[x] = \sum a[i]b[x-i]$. Để tính tích chập của các số phức hoặc của nhiều hơn hai vector: thực hiện FFT, nhân từng phần tử tương ứng, chia cho n, đảo ngược đoạn [start+1, end), rồi thực hiện FFT lần nữa. Phép làm tròn đảm bảo chính xác nếu $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (trong thực tế có thể lên đến 10^{16}; cao hơn với dữ liệu ngẫu nhiên). Nếu không thỏa mãn, hãy dùng NTT/FFTMod.
Time: $\mathcal{O}\left(N \log N\right)$ với $N = |A| + |B|$ ($\sim 1s$ khi $N = 2^{22}$)

be03a2, 42 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C> &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1); // (^ 10% nhanh hơn nếu dùng
      double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n);
    rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i
        / 2];
  }
  vi rev(n);
  rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      rep(j, 0, k) {
        // C z = rt[j + k] * a[i + j + k];
        (nhanh hơn 25% nếu tự viết phép nhân) include-line
        auto x = (double *)&rt[j + k],
        C z(x[0] * y[0] - x[1] * y[1],
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}
vd conv(const vd &a, const vd &b) {
  if (a.empty() || b.empty())
    return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i, 0, sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C &x : in)
    x *= x;
  rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: FFT với độ chính xác cao hơn, có thể dùng để tính tích chập modulo một số nguyên bất kỳ, miễn là $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (trong thực tế có thể lên đến 10^{16} hoặc cao hơn). Các phần tử đầu vào phải thuộc đoạn $[0, \text{mod})$.
Time: $\mathcal{O}\left(N \log N\right)$, với $N = |A| + |B|$ (chậm gấp đôi NTT hoặc FFT thông thường)

"FastFourierTransform.h" b82773, 23 lines

```
typedef vector<ll> vl;
template <int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty())
    return {};
  vl res(sz(a) + sz(b) - 1);
  int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut = int(
      sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i, 0, n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i, 0, sz(res)) {
    ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5)
        ;
    ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
```

```
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) tính $\hat{f}(k) = \sum_x a[x] g^{xk}$ với mọi k, trong đó $g = \text{root}^{(mod-1)/N}$. N phải là lũy thừa của 2. Hữu ích để tính tích chập modulo các số nguyên tố đặc biệt có dạng $2^a b + 1$, trong đó kích thước kết quả tích chập không vượt quá 2^a. Với modulo bất kỳ, xem FFTMod. `conv(a, b) = c`, trong đó $c[x] = \sum a[i] b[x-i]$. Nếu tự tính tích chập: thực hiện NTT trên các đầu vào, nhân từng phần tử tương ứng, chia cho n, đảo ngược đoạn [start+1, end), rồi thực hiện NTT lần nữa. Các phần tử đầu vào phải thuộc $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" d54056, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// Với p < 2^30 còn có thể dùng, chẳng hạn: 5 << 25, 7 << 26,
    479 << 21
// và 483 << 21 (hai giá trị cuối lớn hơn 10^9).
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i, k, 2 * k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      rep(j, 0, k) {
        ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
        a[i + j + k] = ai - z + (z > ai ? mod : 0);
        ai += (ai + z >= mod ? z - mod : z);
      }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty())
    return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), n = 1
      << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i, 0, n) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv
      % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

Polynomial.h

Description: Các phép toán trên chuỗi lũy thừa modulo x^n trên $\mathbb{Z}_{998244353}$. Đa thức được lưu trong vector vl, hệ số có bậc thấp nhất đứng trước. polyMul(a, b): Tính tích hai đa thức (tích chập thông thường). polyInv(a, n): Tìm b sao cho $ab \equiv 1 \pmod{x^n}$, yêu cầu $a[0] \neq 0$. polyLog(a, n): Tính $\ln a \bmod x^n$, yêu cầu $a[0] = 1$. polyExp(a, n): Tính $e^a \bmod x^n$, yêu cầu $a[0] = 0$. Kết quả của các hàm đều có đúng n phần tử (ngoại trừ polyMul).

Usage: `vl f = {1, 1, 1}; vl g = polyInv(f, 8); // g = 1 - x + x^3 - x^4 + ...`

Time: $\mathcal{O}(n \log n)$ cho phép nhân, nghịch đảo và log; exp có hằng số lớn hơn (khoảng gấp 3 lần log). Status: đã kiểm thử trên Library Checker (Inv/Log/Exp of Formal Power Series)

"NumberTheoreticTransform.h" 197b00, 43 lines

```
vl polyMul(const vl &a, const vl &b) { return conv(a, b); }
vl polyInv(const vl &a, int n) {
  vl b = {modpow(a[0], mod - 2)};
  for (int m = 1; m < n; m *= 2) { // b <- b * (2 - a * b) mod
      x^{2m}
    vl ap(a.begin(), a.begin() + min(sz(a), 2 * m));
    vl t = conv(conv(b, b), ap);
    t.resize(2 * m);
    b.resize(2 * m);
    rep(i, 0, 2 * m) b[i] = ((2 * b[i] - t[i]) % mod + mod) %
        mod;
  }
  b.resize(n);
  return b;
}
vl polyDeriv(vl a) {
  rep(i, 1, sz(a)) a[i - 1] = a[i] * i % mod;
  a.pop_back();
  return a;
}
vl polyInteg(vl a) {
  int n = sz(a) + 1;
  vl iv(n + 1, 1); // iv[i] = i^{-1} mod p
  rep(i, 2, n + 1) iv[i] = (mod - (mod / i) * iv[mod % i] % mod
      ) % mod;
  a.insert(a.begin(), 0);
  rep(i, 1, n) a[i] = a[i] * iv[i] % mod;
  return a;
}
vl polyLog(const vl &a, int n) {
  vl r = polyInteg(conv(polyDeriv(a), polyInv(a, n)));
  r.resize(n);
  return r;
}
vl polyExp(const vl &a, int n) {
  vl b = {1};
  for (int m = 1; m < n; m *= 2) { // b <- b * (1 + a - ln b)
      mod x^{2m}
    vl l = polyLog(b, 2 * m), f(2 * m);
    rep(i, 0, 2 * m) f[i] = ((i < sz(a) ? a[i] : 0) - l[i] +
        mod) % mod;
    f[0] = (f[0] + 1) % mod;
    b = conv(b, f);
    b.resize(2 * m);
  }
  b.resize(n);
  return b;
}
```

FastSubsetTransform.h

Description: Biến đổi vector sang một cơ sở cho phép tính tích chập nhanh với công thức: $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, trong đó $\oplus$ là một trong ba phép AND, OR hoặc XOR. Kích thước của vector a phải là lũy thừa của 2.

Time: $\mathcal{O}(N \log N)$

464cf3, 19 lines

```
void FST(vi &a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step)
      rep(j, i, i + step) {
        int &u = a[j], &v = a[j + step];
        tie(u, v) = inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
      }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0);
  FST(b, 0);
  rep(i, 0, sz(a)) a[i] *= b[i];
  FST(a, 1);
  return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Các phép toán số học modulo. Trước tiên cần thiết lập giá trị mod, sau đó có thể sử dụng cấu trúc này.

<cassert>, "euclid.h" ea1ac7, 22 lines

```
const ll mod = 17; // change to something else
struct Mod {
  ll x;
  Mod(ll y) : Mod(y % mod + mod, 0) {}
  Mod(ll y, int) : x(y < mod ? y : y - mod) {}
  Mod operator+(Mod b) { return {x + b.x, 0}; }
  Mod operator-(Mod b) { return {x - b.x + mod, 0}; }
  Mod operator*(Mod b) { return {x * b.x % mod, 0}; }
  Mod operator/(Mod b) { return *this * invert(b); }
  Mod invert(Mod a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1);
    return x;
  }
  Mod operator^(ll e) {
    if (!e)
      return 1;
    Mod r = *this ^ (e / 2);
    r = r * r;
    return e & 1 ? *this * r : r;
  }
};
```

ModInverse.h

Description: Nghịch đảo modulo. Case 1: M nguyên tố (Fermat). Case 2: M bất kỳ, cần $\gcd(a, M) = 1$ (euclid mở rộng). Case 3: nghịch đảo của $1..n$ trong $\mathcal{O}(n)$ khi M nguyên tố, dùng $\text{inv}[i] = -\lfloor M/i \rfloor \cdot \text{inv}[M \bmod i] \bmod M$.

Time: $\mathcal{O}(\log M)$ cho Case 1, 2; $\mathcal{O}(n)$ cho Case 3.

"euclid.h", <cassert> 07837b, 26 lines

```
// Case 1: M is prime.
ll modinvPrime(ll a, ll M) {
  ll r = 1, e = M - 2;
  a %= M;
  if (a < 0)
    a += M;
  for (; e; a = a * a % M, e >>= 1)
    if (e & 1)
      r = r * a % M;
  return r;
}
// Case 2: M is arbitrary. Need gcd(a, M) == 1
ll modinv(ll a, ll M) {
  a = (a % M + M) % M;
  ll x, y, g = euclid(a, M, x, y);
  assert(g == 1); // g != 1 => no inverse
  return (x % M + M) % M;
}

// Case 3: M is prime, need inverses of 1..n (n < M) in O(n).
vector<ll> invln(int n, ll M) {
  vector<ll> inv(n + 1, 1);
  for (int i = 2; i <= n; i++)
    inv[i] = (M - M / i) * inv[M % i] % M;
```

```
  return inv;
}
```

ModPow.h

b83e45, 8 lines

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

Description: Trả về giá trị $x > 0$ nhỏ nhất thỏa mãn $a^x \equiv b \pmod m$, hoặc trả về -1 nếu không tồn tại x như vậy. Có thể sử dụng modLog(a, 1, m) để tính cấp của a modulo m.
Time: $\mathcal{O}\left(\sqrt{m}\right)$

c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m)
    return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i, 2, n + 2) if (A.count(e = e * f % m)) return n * i -
        A[e];
  return -1;
}
```

ModSum.h

Description: Tính tổng của các cấp số cộng sau khi lấy modulo.
modsum(to, c, k, m) $= \sum_{i=0}^{to-1} (ki + c)\%m$. divsum có chức năng tương tự, nhưng sử dụng phép chia lấy phần nguyên. Time: $O(\log m)$, với hằng số khá lớn. Status: Đã kiểm thử với mọi $|k|, |c|, to, m \leq 50$ và trên Kattis: aladin.

5c5bc5, 18 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m;
  c %= m;
  if (!k)
    return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Tính $a \cdot b \bmod c$ (hoặc $a^b \bmod c$) với $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}\left(1\right)$ đối với modmul, $\mathcal{O}\left(\log b\right)$ đối với modpow.

bbbd8f, 12 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1)
      ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Thuật toán Tonelli-Shanks để tìm căn bậc hai modulo. Tìm x thỏa mãn $x^2 \equiv a \pmod p$. Nghiệm còn lại là $-x \pmod p$.
Time: $\mathcal{O}\left(\log^2 p\right)$ trong trường hợp xấu nhất, $\mathcal{O}\left(\log p\right)$ với phần lớn giá trị p.

<cassert> b1cb60, 44 lines

```
// Local modpow that takes explicit modulus (ModPow.h's version
    uses global
// const ll mod)
namespace modsqrt_impl {
ll mpow(ll b, ll e, ll m) {
  ll ans = 1;
  b %= m;
  for (; e; b = b * b % m, e /= 2)
    if (e & 1)
      ans = ans * b % m;
  return ans;
}
} // namespace modsqrt_impl

ll modsqrt(ll a, ll p) {
  using modsqrt_impl::mpow;
  a %= p;
  if (a < 0)
    a += p;
  if (a == 0)
    return 0;
  assert(mpow(a, (p - 1) / 2, p) == 1); // else no solution
  if (p % 4 == 3)
    return mpow(a, (p + 1) / 4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (mpow(n, (p - 1) / 2, p) != p - 1)
    ++n;
  ll x = mpow(a, (s + 1) / 2, p);
  ll b = mpow(a, s, p), g = mpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0)
      return x;
    ll gs = mpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

5.2 Primality

FastEratosthenes.h

Description: Sàng nguyên tố, dùng để tìm tất cả các số nguyên tố nhỏ hơn LIM.
Time: LIM=1e9 $\approx$ 1.5s

6b2912, 24 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S + 1);
  pr.reserve(int(LIM / log(LIM) * 1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2)
    if (!sieve[i]) {
      cp.push_back({i, i * i / 2});
      for (int j = i * i; j <= S; j += 2 * i)
        sieve[j] = 1;
    }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i = idx; i < S + L; idx = (i += p))
        block[i - L] = 1;
    rep(i, 0, min(S, R - L)) if (!block[i]) pr.push_back((L + i
        ) * 2 + 1);
  }
  for (int i : pr)
    isPrime[i] = 1;
  return pr;
}
```

LinearSieve.h

Description: Sàng tuyến tính: mỗi hợp số $i \cdot p$ chỉ bị gạch đúng một lần, bởi ước nguyên tố nhỏ nhất của nó. Tính cho mọi $i \leq$ LIM: lp[i] = ước nguyên tố nhỏ nhất, phi[i] = hàm Euler, mu[i] = hàm Mobius, và pr = danh sách các số nguyên tố tăng dần. Với LIM $= 10^7$ ba mảng int tốn khoảng 120MB, bỏ bớt mảng không cần dùng.
Hàm Euler: $\phi(n)$ = số nguyên dương không vượt quá n và nguyên tố cùng nhau với n. $\phi(1) = 1$, $\phi(p^k) = (p-1)p^{k-1}$ với p nguyên tố, và nhân tính: $\gcd(m,n) = 1 \Rightarrow \phi(mn) = \phi(m)\phi(n)$. Nếu $n = p_1^{k_1} \cdots p_r^{k_r}$ thì $\phi(n) = n\prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$. $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2$ với $n > 1$. Định lý Euler: $\gcd(a,n) = 1 \Rightarrow a^{\phi(n)} \equiv 1 \pmod n$. Định lý nhỏ Fermat: p nguyên tố, $p \nmid a \Rightarrow a^{p-1} \equiv 1 \pmod p$.
Hàm Mobius: $\mu(n) = 0$ nếu n có thừa số chính phương, ngược lại $\mu(n) = (-1)^k$ với k là số ước nguyên tố phân biệt của n. $\sum_{d|n} \mu(d) = [n = 1]$.
Usage: linearSieve(); gọi một lần, sau đó dùng lp, phi, mu, pr với $n \leq$
LIM. Xem ví dụ ở cuối file.
Time: $\mathcal{O}\left(LIM\right)$

873faf, 52 lines

```
const int LIM = 1e6;
int lp[LIM + 1], phi[LIM + 1], mu[LIM + 1];
vi pr;
void linearSieve() {
  phi[1] = mu[1] = 1;
  rep(i, 2, LIM + 1) {
    if (!lp[i])
      lp[i] = i, pr.push_back(i), phi[i] = i - 1, mu[i] = -1;
    for (int p : pr) {
      if (p > lp[i] || (ll)i * p > LIM)
        break;
      lp[i * p] = p;
      if (p == lp[i])
        phi[i * p] = phi[i] * p, mu[i * p] = 0;
      else
        phi[i * p] = phi[i] * (p - 1), mu[i * p] = -mu[i];
    }
  }
}
// Các thừa số nguyên tố của n (có lặp), O(log n)
for (int x = n; x > 1; x /= lp[x])
  cout << lp[x] << ' ';
// Phân tích dạng p^e, và số ước tau(n) = tích (e+1)
int tau = 1;
for (int x = n; x > 1;) {
  int p = lp[x], e = 0;
```

```
  while (x % p == 0)
    x /= p, e++;
  tau *= e + 1; // n có thừa số p^e
}
// Kiểm tra nguyên tố: n >= 2 && lp[n] == n
// Tổng tiền tố phi (dùng ll vì tổng vượt int),mu tương tự
vector<ll> sp(LIM + 1);
rep(i, 1, LIM + 1) sp[i] = sp[i - 1] + phi[i];
// phi của một số n lớn (n <= LIM*LIM, ví dụ LIM = 1e6 thì n <=
      1e12):
ll phiOne(ll n) {
  ll r = n;
  for (int p : pr) {
    if ((ll)p * p > n)
      break;
    if (n % p == 0) {
      while (n % p == 0)
        n /= p;
      r -= r / p;
    }
  }
  if (n > 1)
    r -= r / n;
  return r;
}
// Nghịch đảo modulo m khi biết phi(m): a^(-1) = a^(phi(m)-1)
     mod m, với gcd(a,
// m) = 1.
```

PrimeCounting.h

Description: Đếm số nguyên tố $\le n$, tức $\pi(n)$. Chỉ cần quan tâm các giá trị dạng $\lfloor n/i \rfloor$ (có $2\sqrt{n}$ giá trị). Gọi $S(v,p)$ là số $x \in [2,v]$ mà x là nguyên tố hoặc có ước nguyên tố nhỏ nhất $> p$. Khi đó $S(v,p) = S(v,p-1) - (S(v/p,p-1) - S(p-1,p-1))$ nếu p nguyên tố và $p^2 \le v$, ngược lại giữ nguyên. Bắt đầu từ $S(v,1) = v-1$. lo[i] lưu $S(i,\cdot)$, hi[i] lưu $S(\lfloor n/i \rfloor,\cdot)$. Chỉ cần đổi hàm khởi tạo và phép trừ là đếm được các hàm khác (ví dụ tổng các số nguyên tố, với hàm nhân tính hoàn toàn).
Usage: `ll c = primeCount(1e10);`
Time: $\mathcal{O}\left(n^{3/4}\right)$, bộ nhớ $\mathcal{O}\left(\sqrt{n}\right)$

48703f, 24 lines

```
ll primeCount(ll n) {
  if (n < 2)
    return 0;
  ll r = sqrtl(n);
  while (r * r > n)
    r--;
  while ((r + 1) * (r + 1) <= n)
    r++;
  vector<ll> lo(r + 1), hi(r + 1);
  rep(i, 1, r + 1) lo[i] = i - 1, hi[i] = n / i - 1;
  for (ll p = 2; p <= r; p++) {
    if (lo[p] == lo[p - 1])
      continue; // p không nguyên tố
    ll pc = lo[p - 1], p2 = p * p;
    ll end = min(r, n / p2);
    for (ll i = 1; i <= end; i++) {
      ll d = i * p;
      hi[i] -= (d <= r ? hi[d] : lo[n / d]) - pc;
    }
    for (ll i = r; i >= p2; i--)
      lo[i] -= lo[i / p] - pc;
  }
  return hi[1];
}
```

MillerRabin.h

Description: Thuật toán Miller-Rabin xác định để kiểm tra tính nguyên tố. Đảm bảo hoạt động chính xác với các số không vượt quá $7 \cdot 10^{18}$; với các số lớn hơn, hãy sử dụng Python và mở rộng tập cơ sở A bằng các giá trị được chọn ngẫu nhiên.
Time: Gấp 7 lần độ phức tạp của phép tính $a^b \bmod c$.

"ModMulLL.h" 60dcd1, 14 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1)
    return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
  for (ull a : A) { // ^ count trailing zeroes
    ull p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n - 1 && i != s)
      return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

5.3 Divisibility

euclid.h

Description: Tìm hai số nguyên x và y sao cho $ax + by = \gcd(a,b)$. Nếu chỉ cần tính GCD, hãy sử dụng hàm có sẵn __gcd. Nếu a và b nguyên tố cùng nhau thì x chính là nghịch đảo của a modulo b.

33ba8f, 6 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b)
    return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}
```

CRT.h

Description: Định lý số dư Trung Hoa (Chinese Remainder Theorem). crt(a, m, b, n) tính x sao cho $x \equiv a \pmod m$, $x \equiv b \pmod n$. Nếu $|a| < m$ và $|b| < n$, x sẽ thỏa $0 \le x < \text{lcm}(m,n)$. Giả sử $mn < 2^{62}$.
Time: $\log(n)$

<cassert>, "euclid.h" 04d93a, 8 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m)
    swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m * n / g : x;
}
```

5.3.1 Đẳng thức Bézout

Với $a \neq 0$, $b \neq 0$, đặt $d = \gcd(a,b)$. Khi đó, d là số nguyên dương nhỏ nhất sao cho tồn tại các số nguyên x, y thỏa mãn

$$ax + by = d$$

Nếu (x,y) là một nghiệm thì mọi nghiệm đều có dạng

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

5.4 Combinatorics modulo M

Combinatorics.h

Description: Tính giai thừa, giai thừa nghịch đảo và tổ hợp nCr modulo số nguyên tố M. Gọi initFact() một lần trước khi sử dụng nCr.
Time: $\mathcal{O}(MAXN)$ tiền xử lý, $\mathcal{O}(1)$ cho mỗi truy vấn

"ModInverse.h" c5d503, 15 lines

```
const ll mod = 1000000007;
const int MAXN = 1000001;
ll fact[MAXN], invfact[MAXN];
void initFact() {
  fact[0] = 1;
  rep(i, 1, MAXN) fact[i] = fact[i - 1] * i % mod;
  invfact[MAXN - 1] = modinvPrime(fact[MAXN - 1], mod);
  for (int i = MAXN - 1; i > 0; i--)
    invfact[i - 1] = invfact[i] * i % mod;
}
ll nCr(int n, int r) {
  if (r < 0 || r > n)
    return 0;
  return fact[n] * invfact[r] % mod * invfact[n - r] % mod;
}
```

nCrAny.h

Description: Tính nCr modulo một số nguyên dương M bất kỳ. Phân tích M thành các lũy thừa nguyên tố p^k, tính nCr modulo từng p^k (tính n! sau khi loại bỏ các thừa số p, rồi nhân lại với p^e), sau đó kết hợp kết quả bằng CRT. Mỗi p^k phải không vượt quá khoảng 1e7 (do cần tạo bảng kích thước p^k), và M phải có thể phân tích bằng phương pháp thử chia (M tối đa khoảng 1e12). Nếu M là số nguyên tố lớn, hãy sử dụng nCr trong Combinatorics.h.
Time: $\mathcal{O}\left(sumofp^k + log_p(n)\right)$ per query

"CRT.h", "ModInverse.h", "euclid.h" e7ce2d, 52 lines

```
ll powmod(ll b, ll e, ll m) {
  ll r = 1 % m;
  b %= m;
  for (; e; b = b * b % m, e >>= 1)
    if (e & 1)
      r = r * b % m;
  return r;
}
// n! with all factors of p removed, mod pk
ll factNoP(ll n, ll p, ll pk, const vector<ll> &f) {
  ll res = 1;
  while (n) {
    res = res * powmod(f[pk - 1], n / pk, pk) % pk * f[n % pk]
        % pk;
    n /= p;
```

```
  }
  return res;
}
ll nCrPk(ll n, ll r, ll p, ll k, ll pk) {
  ll e = 0;
  for (ll t = n; t;)
    t /= p, e += t;
  for (ll t = r; t;)
    t /= p, e -= t;
  for (ll t = n - r; t;)
    t /= p, e -= t;
  if (e >= k)
    return 0;
  vector<ll> f(pk);
  f[0] = 1;
  rep(i, 1, pk) f[i] = i % p ? f[i - 1] * i % pk : f[i - 1];
  ll num = factNoP(n, p, pk, f);
  ll den = factNoP(r, p, pk, f) * factNoP(n - r, p, pk, f) % pk
      ;
  return num * modinv(den, pk) % pk * powmod(p, e, pk) % pk;
}
ll nCrAny(ll n, ll r, ll M) {
  if (r < 0 || r > n)
    return 0;
  ll ans = 0, m = 1, x = M;
  for (ll p = 2; p * p <= x; p++)
    if (x % p == 0) {
      ll k = 0, pk = 1;
      while (x % p == 0)
        x /= p, k++, pk *= p;
      ans = crt(ans, m, nCrPk(n, r, p, k, pk), pk);
      m *= pk;
    }
  if (x > 1) {
    ans = crt(ans, m, nCrPk(n, r, x, 1, x), x);
    m *= x;
  }
  return ans % M;
}
```

5.5 Floor blocks

FloorBlocks.h

Description: Duyệt qua các block cực đại $[l, r]$ mà trên đó $\lfloor n/i \rfloor$ không đổi. Có tối đa $2\sqrt{n}$ block. Dùng cho các tổng dạng $\sum_{i=1}^{n} f(i) \cdot \lfloor n/i \rfloor$, với f có tổng tiền tố tính được nhanh (ví dụ $f(i) = i$). Yêu cầu $n \geq 1$.
Usage: Đây là snippet, đặt bên trong hàm.

```
ll n = ...;
for (ll l = 1, r; l <= n; l = r + 1) {
r = n / (n / l);
// với mọi i trong [l, r]:  n / i == n / l
// ví dụ:  ans += (n / l) * (sum(r) - sum(l - 1));
}
```

Time: $\mathcal{O}(\sqrt{n})$ 88ed1f, 4 lines

```
for (ll l = 1, r; l <= n; l = r + 1) {
  r = n / (n / l);
  // for every i in [l, r]: n / i == n / l
}
```

5.6 Fractions

ContinuedFractions.h

Description: Cho trước N và một số thực $x \geq 0$, tìm phân số xấp xỉ hữu tỉ p/q gần với x nhất, với $p, q \leq N$. Kết quả thỏa mãn $|p/q - x| \leq 1/qN$.
Với hai phân số hội tụ liên tiếp, ta có: $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ luân phiên lớn hơn và nhỏ hơn x.)
Nếu x là số hữu tỉ, y cuối cùng sẽ tiến tới ∞. Nếu x là nghiệm của một đa thức bậc 2, các hệ số a cuối cùng sẽ tuần hoàn.

Time: $\mathcal{O}(\log N)$ dd6c5e, 24 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX;
  d y = x;
  for (;;) {
    ll lim = min(P ? (N - LP) / P : inf, Q ? (N - LQ) / Q : inf
        ),
      a = (ll)floor(y), b = min(a, lim), NP = b * P + LP, NQ =
          b * Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q))
          ? make_pair(NP, NQ)
          : make_pair(P, Q);
    }
    if (abs(y = 1 / (y - (d)a)) > 3 * N) {
      return {NP, NQ};
    }
    LP = P;
    P = NP;
    LQ = Q;
    Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Cho trước hàm f và giới hạn N, tìm phân số nhỏ nhất $p/q \in [0, 1]$ sao cho $f(p/q)$ đúng, với $p, q \leq N$. Bạn có thể cho hàm f ném exception nếu tìm được nghiệm chính xác, khi đó có thể bỏ giới hạn N.
Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`
Time: $\mathcal{O}(\log N)$ 27ab3e, 28 lines

```
struct Frac {
  ll p, q;
};
template <class F> Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Đặt hi = 1/0 để tìm kiếm trên kho
      ảng (0, N]
  if (f(lo))
    return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // Di chuyển hi nếu dir, ngược lại di
        chuyển lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step;
        si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B;
    B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.7 Bộ ba Pythagoras

Mọi bộ ba Pythagoras đều được sinh duy nhất bởi

$$a = k \cdot (m^2 - n^2), \;\; b = k \cdot (2mn), \;\; c = k \cdot (m^2 + n^2),$$

với $m > n > 0$, $k > 0$, $m \perp n$ (nguyên tố cùng nhau), và một trong hai số m, n chẵn.

5.8 Số nguyên tố

$p = 962592769$ thỏa $2^{21} \mid p - 1$, có thể hữu ích (ví dụ cho NTT).
Để băm, dùng 970592641 (31 bit), 31443539979727 (45 bit), 3006703054056749 (52 bit). Có 78498 số nguyên tố nhỏ hơn 1 000 000.

Căn nguyên thủy tồn tại modulo mọi lũy thừa nguyên tố p^a, trừ $p = 2, a > 2$, và có đúng $\phi(\phi(p^a))$ căn nguyên thủy. Với $p = 2, a > 2$, nhóm $\mathbb{Z}_{2^a}^{\times}$ đẳng cấu với $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.9 Ước lượng

$\sum_{d|n} d = O(n \log \log n)$.

Số ước của n nhiều nhất khoảng 100 khi $n < 5 \cdot 10^4$, 500 khi $n < 10^7$, 2000 khi $n < 10^{10}$, 200 000 khi $n < 10^{19}$.

5.10 Hàm Mobius

$$\mu(n) = \begin{cases} 0 & n \text{ có ước chính phương} \\ 1 & n \text{ có số lượng ước nguyên tố chẵn} \\ -1 & n \text{ có số lượng ước nguyên tố lẻ} \end{cases}$$

(Số ước nguyên tố tính theo các ước nguyên tố phân biệt, n không có ước chính phương.)

Công thức nghịch đảo Mobius:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Một số công thức hữu ích khác:

$\sum_{d|n} \mu(d) = [n = 1]$ (rất hay dùng)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorics (6)

6.1 Hoán vị

6.1.1 Giai thừa

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

6.1.2 Hoán vị không điểm cố định (số lệch)

Hoán vị của một tập sao cho không phần tử nào nằm ở vị trí ban đầu. Ký hiệu $\lfloor x \rceil$ là làm tròn đến số nguyên gần nhất.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.3 Bổ đề Burnside

Cho nhóm các phép đối xứng G tác động lên tập X. Số phần tử của X *xét tới đối xứng* bằng

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

trong đó X^g là tập các phần tử bị g giữ cố định ($g.x = x$).

Nếu $f(n)$ đếm số cấu hình có độ dài n, bỏ qua đối xứng quay ($G = \mathbb{Z}_n$):

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

6.2 Phân hoạch

6.2.1 Hàm phân hoạch số

Số cách viết n thành tổng các số nguyên dương, không tính thứ tự.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.3 Các dãy số thường gặp

6.3.1 Số Stirling loại 1

Số hoán vị của n phần tử có đúng k chu trình.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

6.3.2 Số Stirling loại 2

Số cách phân hoạch n phần tử phân biệt thành đúng k nhóm không phân biệt thứ tự.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k),\ S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.3 Số Bell

Tổng số cách phân hoạch n phần tử phân biệt:
$B(n) = \sum_k S(n,k)$.
$B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$

6.3.4 Cây không gốc có gán nhãn

Số cây trên n đỉnh: n^{n-2}
Số cách nối k cây có sẵn (kích thước n_i) thành một cây: $n_1 n_2 \cdots n_k n^{k-2}$
Số cây với bậc các đỉnh là d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.5 Số Catalan

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- Số đường đi đơn điệu nằm dưới đường chéo trong lưới $n \times n$.
- Số dãy ngoặc đúng gồm n cặp ngoặc.
- Số cây nhị phân có $n+1$ lá, mỗi nút có 0 hoặc 2 con.
- Số cây có thứ tự gồm $n+1$ đỉnh.
- Số cách tam giác hóa đa giác lồi $n+2$ cạnh.
- Số hoán vị của $[n]$ không chứa dãy con tăng độ dài 3.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Tìm đường đi ngắn nhất từ đỉnh s trên đồ thị có thể chứa cạnh trọng số âm. Đỉnh không thể đi tới có dist = inf; đỉnh có thể đi tới thông qua chu trình trọng số âm có dist = -inf. Giả sử $V^2 \max|w_i| < 2^{63}$.
Time: $\mathcal{O}(VE)$
830a8f, 28 lines

```
const ll inf = LLONG_MAX;
struct Ed {
  int a, b, w, s() { return a < b ? a : -a; }
};
struct Node {
  ll dist = inf;
  int prev = -1;
};
void bellmanFord(vector<Node> &nodes, vector<Ed> &eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i, 0, lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf)
      continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim - 1 ? d : -inf);
    }
  }
  rep(i, 0, lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

FloydWarshall.h

Description: Tìm đường đi ngắn nhất giữa mọi cặp đỉnh trên đồ thị có hướng, có thể chứa cạnh trọng số âm. Đầu vào là ma trận khoảng cách m, trong đó m[i][j] = inf nếu không có cạnh trực tiếp từ i đến j. Sau khi chạy: - m[i][j] = khoảng cách ngắn nhất từ i đến j. - m[i][j] = inf nếu không tồn tại đường đi. - m[i][j] = -inf nếu có thể đi qua chu trình trọng số âm.
Time: $\mathcal{O}(N^3)$
531245, 12 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>> &m) {
  int n = sz(m);
  rep(i, 0, n) m[i][i] = min(m[i][i], 0LL);
  rep(k, 0, n) rep(i, 0, n)
    rep(j, 0, n) if (m[i][k] != inf && m[k][j] != inf) {
    auto newDist = max(m[i][k] + m[k][j], -inf);
    m[i][j] = min(m[i][j], newDist);
  }
  rep(k, 0, n) if (m[k][k] < 0) rep(i, 0, n)
    rep(j, 0, n) if (m[i][k] != inf && m[k][j] != inf) m[i][j]
        = -inf;
}
```

TopoSort.h

Description: Sắp xếp topo trên đồ thị có hướng. Kết quả là thứ tự các đỉnh sao cho mọi cạnh đều hướng từ trái sang phải. Nếu đồ thị có chu trình, danh sách trả về sẽ có ít hơn n đỉnh; các đỉnh có thể đi tới từ chu trình cũng sẽ không được trả về.
Time: $\mathcal{O}(V+E)$
d678d8, 9 lines

```
vi topoSort(const vector<vi> &gr) {
  vi indeg(sz(gr)), q;
  for (auto &li : gr)
    for (int x : li)
      indeg[x]++;
  rep(i, 0, sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j, 0, sz(q)) for (int x : gr[q[j]]) if (--indeg[x] == 0)
      q.push_back(x);
  return q;
}
```

Dijkstra.h

Description: Tìm đường đi ngắn nhất từ đỉnh s trên đồ thị có trọng số cạnh không âm. Trả về dist, par, trong đó: - dist[i]: khoảng cách ngắn nhất từ s đến i (inf nếu không thể đi tới). - par[i]: đỉnh đứng trước i trên đường đi ngắn nhất (hoặc -1 nếu không có).
Usage: `auto [dist, par] = dijkstra(s, adj);`
Time: $\mathcal{O}((V+E)logV)$
2c7e1a, 28 lines

```
const ll inf = 1LL << 62;
pair<vector<ll>, vi> dijkstra(int s, const vector<vector<pair<
    int, ll>>> &adj) {
  int n = sz(adj);
  vector<ll> dist(n, inf);
  vi par(n, -1);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<
      pair<ll, int>>>
    pq;
  dist[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    pair<ll, int> cur = pq.top();
    ll d = cur.first;
    int u = cur.second;
    pq.pop();
    if (d != dist[u])
      continue;
```

```
    for (const pair<int, ll> &e : adj[u]) {
      int v = e.first;
      ll w = e.second;
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        par[v] = u;
        pq.push({dist[v], v});
      }
    }
  }
  return {dist, par};
}
```

Kruskal.h

Description: Tìm cây khung nhỏ nhất (MST) trên đồ thị vô hướng có trọng số bằng thuật toán Kruskal kết hợp cấu trúc Disjoint Set Union (UF). Trả về cặp {tổng trọng số MST, danh sách chỉ số các cạnh thuộc MST}. Nếu đồ thị không liên thông, thuật toán trả về rừng khung nhỏ nhất (MSF); kiểm tra `sz(mst) == n - 1` để xác nhận đồ thị liên thông.
Usage: `vector<Edge> ed;`
`rep(i, 0, M) ed.push_back({u, v, w, i});`
`auto [cost, mst] = kruskal(N, ed);`
Time: $\mathcal{O}(E \log E)$

d9afe5, 34 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b)
      return false;
    if (e[a] > e[b])
      swap(a, b);
    e[a] += e[b];
    e[b] = a;
    return true;
  }
};
struct Edge {
  int u, v;
  ll w;
  int id;
  bool operator<(const Edge &o) const { return w < o.w; }
};

pair<ll, vi> kruskal(int n, vector<Edge> &ed) {
  sort(all(ed));
  UF uf(n);
  ll cost = 0;
  vi mst;
  for (auto &e : ed)
    if (uf.join(e.u, e.v)) {
      cost += e.w;
      mst.push_back(e.id);
    }
  return {cost, mst};
}
```

ShortestCycle.h

Description: Tìm độ dài chu trình ngắn nhất (girth) trong đồ thị vô hướng không trọng số. Trả về -1 nếu đồ thị không có chu trình.
Usage: `int len = shortestCycle(adj);`
Time: $\mathcal{O}(V(V+E))$

e29234, 19 lines

```
int shortestCycle(const vector<vi> &adj) {
  int n = sz(adj), ans = INT_MAX;
  rep(s, 0, n) {
    vi dist(n, -1), par(n, -1), q = {s};
    dist[s] = 0;
    rep(qi, 0, sz(q)) {
      int u = q[qi];
      for (int v : adj[u]) {
        if (dist[v] == -1) {
          dist[v] = dist[u] + 1;
          par[v] = u;
          q.push_back(v);
        } else if (par[u] != v)
          ans = min(ans, dist[u] + dist[v] + 1);
      }
    }
  }
  return ans == INT_MAX ? -1 : ans;
}
```

7.2 Network flow

MinCostMaxFlow.h

Description: Tìm luồng cực đại có chi phí nhỏ nhất (Min-Cost Max-Flow). Nếu đồ thị có cạnh mang chi phí âm, cần gọi setpi trước maxflow. Lưu ý: Không hỗ trợ chu trình có tổng chi phí âm. Để lấy luồng thực tế trên các cạnh, chỉ xét những cạnh có giá trị luồng dương.
Time: $\mathcal{O}(FElogV)$, với F là giá trị luồng cực đại. Hàm setpi có độ phức tạp $\mathcal{O}(VE)$.

<bits/extc++.h>, <cassert> 135b73, 79 lines

```
const ll INF = numeric_limits<ll>::max() / 4;
struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge *> par;
  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}
  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to)
      return;
    ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0}
        );
    ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost,
         0});
  }
  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0;
    ll di;
    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({0, s});

    while (!q.empty()) {
      s = q.top().second;
      q.pop();
      seen[s] = 1;
      di = dist[s] + pi[s];
      for (edge &e : ed[s])
        if (!seen[e.to]) {
          ll val = di - pi[e.to] + e.cost;
          if (e.cap - e.flow > 0 && val < dist[e.to]) {
            dist[e.to] = val;
            par[e.to] = &e;
            if (its[e.to] == q.end())
              its[e.to] = q.push({-dist[e.to], e.to});
            else
              q.modify(its[e.to], {-dist[e.to], e.to});
          }
        }
    }
    rep(i, 0, N) pi[i] = min(pi[i] + dist[i], INF);
  }
  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge *x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge *x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    rep(i, 0, N) for (edge &e : ed[i]) totcost += e.cost * e.
        flow;
    return {totflow, totcost / 2};
  }
  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while (ch-- && it--)
      rep(i, 0,
        N) if (pi[i] !=
            INF) for (edge &e :
                ed[i]) if (e.cap) if ((v = pi[i] + e.cost) <
                            pi[e.to]) pi[e.to] = v,
                                ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};
```

Dinic.h

Description: Tìm luồng cực đại bằng thuật toán Dinic. Độ phức tạp: $O(VE \log U)$, với $U = \max|\text{cap}|$. Nếu $U = 1$ thì độ phức tạp là $O(\min(E^{1/2}, V^{2/3})E)$. Với bài toán ghép cặp hai phía thì là $O(\sqrt{V}E)$.

df10f1, 47 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f)
      return f;
    for (int &i = ptr[v]; i < sz(adj[v]); i++) {
      Edge &e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
```

```
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0;
    q[0] = s;
    rep(L, 0, 31) do { // 'int L=30' maybe faster for random
        data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX))
        flow += p;
    }
    while (lvl[t])
      ;
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: Sau khi chạy luồng cực đại (max-flow), phía trái của lát cắt nhỏ nhất từ s đến t gồm tất cả các đỉnh có thể đi tới được từ s, chỉ đi qua các cạnh còn dung lượng dư (residual capacity) dương. Trong Dinic và PushRelabel đã có sẵn hàm `leftOfMinCut(a)` để kiểm tra đỉnh a có thuộc phía trái hay không.

GlobalMinCut.h

Description: Tìm lát cắt nhỏ nhất toàn cục (global minimum cut) trong đồ thị vô hướng, đồ thị cho dưới dạng ma trận kề (`mat[i][j]` là trọng số cạnh giữa i và j). Trả về cặp {tổng trọng số của lát cắt, danh sách các đỉnh thuộc một phía của lát cắt}.

Time: $\mathcal{O}\left(V^3\right)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i, 0, n) co[i] = {i};
  rep(ph, 1, n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it, 0, n - ph) { // O(V^2) -> O(E log V) with prio.
        queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i, 0, n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i, 0, n) mat[s][i] += mat[t][i];
    rep(i, 0, n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

7.3 Matching

HopcroftKarp.h

Description: Tìm ghép cặp cực đại trên đồ thị hai phía bằng thuật toán Hopcroft-Karp. g[u] chứa danh sách các đỉnh kề thuộc phía phải của đỉnh u ở phía trái. r phải được khởi tạo toàn bộ bằng -1 và có kích thước bằng số đỉnh phía phải. Trả về kích thước matching. r[v] là đỉnh phía trái được ghép với v, hoặc -1 nếu v chưa được ghép.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 23 lines

```
int hopcroftKarp(vector<vi> &g, vi &r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i, 0, t) for (int v : g[q[i]]) {
      if (r[v] == -1)
        f = 1;
      else if (!d[r[v]])
        d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f)
      return res;
    rep(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

DFSMatching.h

Description: Thuật toán đơn giản tìm ghép cặp cực đại trên đồ thị hai phía. g[u] chứa danh sách các đỉnh kề thuộc phía phải của đỉnh u ở phía trái. btoa phải được khởi tạo toàn bộ bằng -1 và có kích thước bằng số đỉnh phía phải. Trả về kích thước matching. btoa[v] là đỉnh phía trái được ghép với v, hoặc -1 nếu v chưa được ghép.

Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`

Time: $\mathcal{O}\left(VE\right)$

522b98, 24 lines

```
bool find(int j, vector<vi> &g, vi &btoa, vi &vis) {
  if (btoa[j] == -1)
    return 1;
  vis[j] = 1;
  int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi> &g, vi &btoa) {
  vi vis;
  rep(i, 0, sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Tìm minimum vertex cover trên đồ thị hai phía. Kích thước của tập phủ bằng kích thước maximum matching. Phần bù của minimum vertex cover là maximum independent set.

<cassert>, "DFSMatching.h" da4196, 24 lines

```
vi cover(vector<vi> &g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match)
    if (it != -1)
      lfound[it] = false;
  vi q, cover;
  rep(i, 0, n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back();
    q.pop_back();
    lfound[i] = 1;
    for (int e : g[i])
      if (!seen[e] && match[e] != -1) {
        seen[e] = true;
        q.push_back(match[e]);
      }
  }
  rep(i, 0, n) if (!lfound[i]) cover.push_back(i);
  rep(i, 0, m) if (seen[i]) cover.push_back(n + i);
  assert(sz(cover) == res);
  return cover;
}
```

7.4 DFS algorithms

SCC.h

Description: Tìm các thành phần liên thông mạnh (SCC) trong đồ thị có hướng. Hai đỉnh thuộc cùng một SCC khi có thể đi từ mỗi đỉnh đến đỉnh còn lại. Hàm scc(graph, callback) gọi callback với danh sách các đỉnh thuộc từng SCC, theo thứ tự topo ngược. comp[v] là chỉ số SCC chứa v; mỗi SCC chỉ có cạnh đến các SCC có chỉ số nhỏ hơn. ncomps là tổng số SCC.

Time: $\mathcal{O}\left(V+E\right)$

76b5c9, 29 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template <class G, class F> int dfs(int j, G &g, F &f) {
  int low = val[j] = ++Time, x;
  z.push_back(j);
  for (auto e : g[j])
    if (comp[e] < 0)
      low = min(low, val[e] ?: dfs(e, g, f));

  if (low == val[j]) {
    do {
      x = z.back();
      z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont);
    cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template <class G, class F> void scc(G &g, F f) {
  int n = sz(g);
  val.assign(n, 0);
  comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i, 0, n) if (comp[i] < 0) dfs(i, g, f);
}
```

BiconnectedComponents.h

Description: Tìm tất cả các thành phần liên thông hai đỉnh trong đồ thị vô hướng và gọi callback với danh sách cạnh của từng thành phần. Trong một thành phần, giữa hai đỉnh bất kỳ có ít nhất hai đường đi không có đỉnh trong chung. Một đỉnh có thể thuộc nhiều thành phần. Cạnh không thuộc thành phần nào là cầu, tức không nằm trên chu trình.
Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`
Time: $\mathcal{O}(V+E)$

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template <class F> int dfs(int at, int par, F &f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at])
    if (e != par) {
      if (num[y]) {
        top = min(top, num[y]);
        if (num[y] < me)
          st.push_back(e);
      } else {
        int si = sz(st);
        int up = dfs(y, e, f);
        top = min(top, up);
        if (up == me) {
          st.push_back(e);
          f(vi(st.begin() + si, st.end()));
          st.resize(si);
        } else if (up < me)
          st.push_back(e);
        else { /* e is a bridge */
        }
      }
    }
  return top;
}

template <class F> void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i, 0, sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

2sat.h

Description: Giải bài toán 2-SAT: cho N biến boolean và các mệnh đề dạng "a hoặc b", tìm một phép gán thỏa mãn tất cả mệnh đề hoặc kết luận vô nghiệm. Biến đánh số từ 0. Literal "biến x đúng" viết là `x`, literal "biến x sai" viết là phép phủ định bit của x (xem Usage). `either(a, b)` thêm mệnh đề $a \vee b$. `setValue(x)` ép literal `x` đúng. `atMostOne(li)` thêm ràng buộc có tối đa một literal trong `li` đúng (tạo thêm biến phụ, mất $\mathcal{O}(|li|)$). `solve()` trả về true nếu có nghiệm, khi đó `values[i]` là giá trị của biến i. Cách làm: mệnh đề $a \vee b$ cho hai cạnh kéo theo $\neg a \to b$ và $\neg b \to a$, rồi tìm SCC bằng Tarjan. Vô nghiệm khi và chỉ khi có biến x mà x và $\neg x$ thuộc cùng một SCC.
Usage: `TwoSat ts(n); // n biến`
`ts.either(0,  3); // biến 0 đúng hoặc biến 3 sai`
`ts.setValue(2); // biến 2 đúng`
`ts.atMostOne({0,  1, 2}); // tối đa một trong ba literal đúng`
`ts.solve(); // true nếu có nghiệm, kết quả ở ts.values[0..n-1]`
Time: $\mathcal{O}(N+E)$

ebb943, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true
  TwoSat(int n = 0) : N(n), gr(2 * n) {}
  int addVar() { // new auxiliary variable
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }
  void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
  }
  void setValue(int x) { either(x, x); }
  void atMostOne(const vi &li) {
    if (sz(li) <= 1)
      return;
    int cur = ~li[0];
    rep(i, 2, sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }
  vi val, comp, z;
  int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x;
    z.push_back(i);
    for (int e : gr[i])
      if (!comp[e])
        low = min(low, val[e] ?: dfs(e));
    if (low == val[i])
      do {
        x = z.back(), z.pop_back();
        comp[x] = low;
        if (values[x >> 1] == -1)
          values[x >> 1] = x & 1;
      } while (x != i);
    return val[i] = low;
  }
  bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0);
    comp = val;
    rep(i, 0, 2 * N) if (!comp[i]) dfs(i);
    rep(i, 0, N) if (comp[2 * i] == comp[2 * i + 1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

Description: Tìm đường đi hoặc chu trình Euler trong đồ thị có hướng hoặc vô hướng bằng thuật toán Hierholzer. Đồ thị được biểu diễn bằng danh sách các cặp (đỉnh đến, ID cạnh toàn cục). Với đồ thị vô hướng, hai chiều của cùng một cạnh phải có chung ID. Trả về dãy đỉnh trên đường đi Euler, hoặc dãy rỗng nếu không tồn tại. Nếu cần lấy ID cạnh, lưu thêm .second vào stack s và kết quả ret.
Time: $\mathcal{O}(V+E)$

780b64, 23 lines

```
vi eulerWalk(vector<vector<pii>> &gr, int nedges, int src = 0)
    {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end) {
      ret.push_back(x);
      s.pop_back();
      continue;
    }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1;
      s.push_back(y);
    }
  }
  for (int x : D)
    if (x < 0 || sz(ret) != nedges + 1)
      return {};
  return {ret.rbegin(), ret.rend()};
}
```

7.5 Trees

BinaryLifting.h

Description: Tiền xử lý các tổ tiên cách nhau 2^k cạnh của mỗi đỉnh trong cây, hỗ trợ nhảy lên nhanh và tìm LCA. Giả sử đỉnh gốc là tổ tiên của chính nó.
Time: Tiền xử lý $\mathcal{O}(N \log N)$, mỗi truy vấn $\mathcal{O}(\log N)$

bfce85, 27 lines

```
vector<vi> treeJump(vi &P) {
  int on = 1, d = 1;
  while (on < sz(P))
    on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i, 1, d) rep(j, 0, sz(P)) jmp[i][j] = jmp[i - 1][jmp[i -
      1][j]];
  return jmp;
}

int jmp(vector<vi> &tbl, int nod, int steps) {
  rep(i, 0, sz(tbl)) if (steps & (1 << i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi> &tbl, vi &depth, int a, int b) {
  if (depth[a] < depth[b])
    swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b)
    return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d)
      a = c, b = d;
  }
  return tbl[0][a];
}
```

LCA.h

Description: Cấu trúc dữ liệu tìm tổ tiên chung thấp nhất (LCA) trên cây với đỉnh 0 làm gốc. C là danh sách kề của cây, có thể là đồ thị có hướng hoặc vô hướng.
Time: Tiền xử lý $\mathcal{O}(N \log N)$, mỗi truy vấn $\mathcal{O}(1)$

"../data-structures/RMQ.h" 0f62fb, 21 lines

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;
  LCA(vector<vi> &C) : time(sz(C)), rmq((dfs(C, 0, -1), ret)) {
      }
  void dfs(vector<vi> &C, int v, int par) {
    time[v] = T++;
    for (int y : C[v])
      if (y != par) {
```

```
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }
  int lca(int a, int b) {
    if (a == b)
      return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  // dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

HLD1.h

Description: Heavy-Light Decomposition. Tách cây có gốc thành các chuỗi (chain) sao cho mọi đường đi từ đỉnh lên gốc cắt $\mathcal{O}(\log N)$ chuỗi. Mỗi chuỗi là một đoạn liên tục trong mảng pos, nên truy vấn đường đi hoặc cây con quy về vài truy vấn đoạn trên Segment Tree. Đỉnh con nặng (heavy child) là con có cây con lớn nhất và được đánh số liền sau cha, nên cây con của v chiếm đúng đoạn $[\text{pos}[v], \text{pos}[v]+\text{siz}[v])$. Giá trị nằm trên đỉnh: update(v, x) gán giá trị đỉnh v, queryPath(u, v) trả về f trên mọi đỉnh của đường đi từ u tới v, querySubtree(v) tương tự cho cây con của v. Giá trị nằm trên cạnh: lưu giá trị cạnh $(\text{par}[v], v)$ tại đỉnh con v, gọi update(v, x) với v là đỉnh con, và truyền edges = true cho query (bỏ qua đỉnh LCA, vì LCA không có cạnh lên). Hàm gộp f (đặt trong Tree) phải có tính kết hợp và giao hoán (max, min, sum, xor, ...). Nếu f không giao hoán thì phải xử lý thứ tự hai nửa đường đi (lên từ u và xuống tới v) riêng. Muốn cộng/gán trên cả đường đi hoặc cây con thì thay Tree bằng Lazy Segment Tree với cùng cách chia đoạn. DFS viết lặp nên không tràn stack với cây lớn. Đỉnh đánh số từ 0, mặc định gốc là 0. vals (nếu có) là giá trị ban đầu của từng đỉnh. Dựng cây mất $\mathcal{O}(N \log N)$, query cây con và lca mất $\mathcal{O}(\log N)$.

Usage: HLD h(g, vals); // g: danh sách kề, vals[i]: giá trị đỉnh i
h.update(v, 5); h.queryPath(u, v); h.querySubtree(v); h.lca(u, v);

Time: Query đường đi $\mathcal{O}(\log^2 N)$.

"../data-structures/SegmentTree.h" fe4dda, 62 lines

```
struct HLD {
  typedef Tree::T T;
  int n, timer = 0;
  vi par, siz, dep, head, pos, heavy;
  Tree seg;
  HLD(const vector<vi> &g, vector<T> vals = {}, int root = 0)
    : n(sz(g)), par(n, -1), siz(n, 1), dep(n), head(n), pos(n),
      heavy(n, -1), seg(n) {
    vi ord, st = {root};
    while (!st.empty()) { // DFS lặp: tính cha, độ sâu
      int v = st.back();
      st.pop_back();
      ord.push_back(v);
      for (int u : g[v])
        if (u != par[v])
          par[u] = v, dep[u] = dep[v] + 1, st.push_back(u);
    }
    for (int i = n; i--;) { // cộng dồn size từ dưới lên, chọn con nặng
      int v = ord[i], p = par[v];
      if (p >= 0) {
        siz[p] += siz[v];
        if (heavy[p] < 0 || siz[v] > siz[heavy[p]])
          heavy[p] = v;
      }
    }
    st.assign(1,
          root); // đánh số: con nhẹ push trước, con nặng push sau cùng
    while (!st.empty()) {
      int v = st.back();
      st.pop_back();
      pos[v] = timer++;
      head[v] = (par[v] >= 0 && heavy[par[v]] == v) ? head[par[
          v]] : v;
      for (int u : g[v])
        if (u != par[v] && u != heavy[v])
          st.push_back(u);
      if (heavy[v] >= 0)
        st.push_back(heavy[v]);
    }
    rep(v, 0, n) if (v < sz(vals)) seg.update(pos[v], vals[v]);
  }
  void update(int v, T x) { seg.update(pos[v], x); }
  T queryPath(int u, int v, bool edges = false) {
    T res = Tree::unit;
    for (; head[u] != head[v]; v = par[head[v]]) {
      if (dep[head[u]] > dep[head[v]])
        swap(u, v);
      res = seg.f(res, seg.query(pos[head[v]], pos[v] + 1));
    }
    if (dep[u] > dep[v])
      swap(u, v);
    return seg.f(res, seg.query(pos[u] + edges, pos[v] + 1));
  }
  T querySubtree(int v, bool edges = false) {
    return seg.query(pos[v] + edges, pos[v] + siz[v]);
  }
  int lca(int u, int v) {
    for (; head[u] != head[v]; v = par[head[v]])
      if (dep[head[u]] > dep[head[v]])
        swap(u, v);
    return dep[u] < dep[v] ? u : v;
  }
};
```

Geometry (8)

8.1 Các phép toán hình học cơ bản

Point.h

Description: Cấu trúc điểm trong mặt phẳng 2D. T có thể là double hoặc long long (tránh dùng int).

47ec0a, 29 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template <class T> struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y);
      }
  bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y
      ); }
  P operator+(P p) const { return P(x + p.x, y + p.y); }
  P operator-(P p) const { return P(x - p.x, y - p.y); }
  P operator*(T d) const { return P(x * d, y * d); }
  P operator/(T d) const { return P(x / d, y / d); }
  T dot(P p) const { return x * p.x + y * p.y; }
  T cross(P p) const { return x * p.y - y * p.x; }
  T cross(P a, P b) const { return (a - *this).cross(b - *this)
      ; }
  T dist2() const { return x * x + y * y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this / dist(); } // makes dist()=1
  P perp() const { return P(-y, x); }     // rotates +90
      degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
  }
  friend ostream &operator<<(ostream &os, P p) {
    return os << "(" << p.x << "," << p.y << ")";
  }
};
```

sideOf.h

Description: Xác định vị trí của điểm p khi nhìn từ s theo hướng đến e. Trả về 1/0/-1 tương ứng với: bên trái / trên đường thẳng / bên phải. Nếu có tham số tùy chọn *eps*, trả về 0 nếu p cách đường thẳng một khoảng không quá *eps*. P được giả định là Point<T>, trong đó T có thể là double hoặc long long. Các phép nhân trung gian có thể gây tràn số khi sử dụng int hoặc long long.

Usage: bool left = sideOf(p1,p2,q)==1;

"Point.h" ca4665, 7 lines

```
template <class P> int sideOf(P s, P e, P p) { return sgn(s.
    cross(e, p)); }

template <class P> int sideOf(P s, P e, P p, double eps) {
  auto a = (e - s).cross(p - s);
  double l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

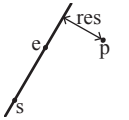
Description: Kiểm tra điểm p có nằm trên đoạn thẳng se hay không. Với Point<double>, nên dùng (segDist(s,e,p)<=epsilon) thay thế.

"Point.h" c597e8, 3 lines

```
template <class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

lineDistance.h

Description:
Tính khoảng cách có dấu từ điểm p đến đường thẳng đi qua hai điểm a và b. Giá trị dương nếu p nằm bên trái và âm nếu nằm bên phải khi nhìn theo hướng từ a đến b. Nếu $a = b$, kết quả là nan. P được giả định là Point<T> hoặc Point3D<T>, trong đó T có thể là double hoặc long long. Các phép nhân trung gian có thể gây tràn số khi sử dụng int hoặc long long. Với Point3D, kết quả luôn không âm. Khi đó, gọi .dist() trên kết quả của tích có hướng để lấy khoảng cách.

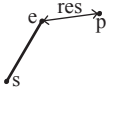


"Point.h" 53630a, 3 lines

```
template <class P> double lineDist(P a, P b, P p) {
  return (double)(b - a).cross(p - a) / (b - a).dist();
}
```

SegmentDistance.h

Description:
Tính khoảng cách ngắn nhất từ điểm p đến đoạn thẳng nối hai điểm s và e.



Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

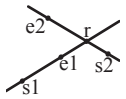
"Point.h" d5f39f, 7 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s == e)
    return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e -
      s)));
  return ((p - s) * d - (e - s) * t).dist() / d;
}
```

lineIntersection.h

Description:

Nếu hai đường thẳng đi qua s_1, e_1 và s_2, e_2 có giao điểm duy nhất, trả về $\{1, \text{điểm giao}\}$. Nếu không có giao điểm, trả về $\{0, (0,0)\}$. Nếu có vô số giao điểm (hai đường thẳng trùng nhau), trả về $\{-1, (0,0)\}$. Nếu P là Point<ll> và giao điểm không có tọa độ nguyên, kết quả trả về sẽ không chính xác. Các phép tính trung gian sử dụng tích của ba tọa độ, cần chú ý tràn số khi dùng int hoặc ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`

```
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

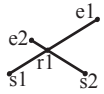
"Point.h" a01f81, 7 lines

```
template <class P> pair<int, P> lineInter(P s1, P e1, P s2, P
    e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

SegmentIntersection.h

Description:

Tìm giao điểm của hai đoạn thẳng s_1e_1 và s_2e_2. Nếu hai đoạn thẳng có đúng một giao điểm, trả về vector chứa điểm giao đó. Nếu không có giao điểm, trả về vector rỗng. Nếu có vô số giao điểm, trả về vector gồm 2 phần tử là hai đầu mút của đoạn thẳng chung. Nếu P là Point<ll> và giao điểm không có tọa độ nguyên, kết quả trả về sẽ không chính xác. Các phép tính trung gian sử dụng tích của ba tọa độ, cần chú ý tràn số khi dùng int hoặc long long.



Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`

```
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"OnSegment.h", "Point.h" 9d57f2, 17 lines

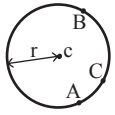
```
template <class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b), oc = a.cross(b,
      c),
    od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a))
    s.insert(a);
  if (onSegment(c, d, b))
    s.insert(b);
  if (onSegment(a, b, c))
    s.insert(c);
  if (onSegment(a, b, d))
    s.insert(d);
  return {all(s)};
}
```

8.2 Hình tròn

circumcircle.h

Description:

Đường tròn ngoại tiếp tam giác là đường tròn đi qua cả ba đỉnh của tam giác. ccRadius trả về bán kính đường tròn ngoại tiếp đi qua ba điểm A, B, C. ccCenter trả về tâm của đường tròn ngoại tiếp đó.



"Point.h" 6cffc5, 9 lines

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B - A).dist() * (C - B).dist() * (A - C).dist() /
      abs((B - A).cross(C - A)) / 2;
}
P ccCenter(P A, P B, P C) {
  P b = C - A, c = B - A;
  return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c
      ) / 2;
}
```

CircleIntersection.h

Description: Tính hai giao điểm của hai đường tròn. Trả về false nếu hai đường tròn không giao nhau.

<cassert>, "Point.h" 84d6d3, 15 lines

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1, double r2, pair<P, P> *
    out) {
  if (a == b) {
    assert(r1 != r2);
    return false;
  }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
      p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 * r1 -
          p * p * d2;
  if (sum * sum < d2 || dif * dif > d2)
    return false;
  P mid = a + vec * p, per = vec.perp() * sqrt(fmax(0, h2) / d2
      );
  *out = {mid + per, mid - per};
  return true;
}
```

8.3 Đa giác

PolygonArea.h

Description: Tính hai lần diện tích có dấu của đa giác. Các đỉnh được liệt kê theo chiều kim đồng hồ cho kết quả âm. Cần chú ý tràn số nếu T là int.

"Point.h" f12300, 5 lines

```
template <class T> T polygonArea2(vector<Point<T>> &v) {
  T a = v.back().cross(v[0]);
  rep(i, 0, sz(v) - 1) a += v[i].cross(v[i + 1]);
  return a;
}
```

InsidePolygon.h

Description: Kiểm tra điểm p có nằm bên trong đa giác hay không. Nếu strict = true, trả về false đối với điểm nằm trên biên đa giác. Thuật toán sử dụng các phép nhân trung gian nên cần chú ý tràn số.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`

```
bool in = inPolygon(v, P{3, 3}, false);
```

Time: $\mathcal{O}(n)$

"OnSegment.h", "Point.h", "SegmentDistance.h" 2bf504, 11 lines

```
template <class P> bool inPolygon(vector<P> &p, P a, bool
    strict = true) {
  int cnt = 0, n = sz(p);
  rep(i, 0, n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a))
      return !strict;
    // or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q) >
        0;
  }
  return cnt;
}
```

ConvexHull.h

Description:

Trả về vector chứa các điểm thuộc bao lồi (convex hull) theo thứ tự ngược chiều kim đồng hồ. Các điểm nằm trên cạnh bao lồi, giữa hai đỉnh khác, không được xem là đỉnh của bao lồi.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 15 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1)
    return pts;
  sort(all(pts));
  vector<P> h(sz(pts) + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0)
        t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

8.4 Các bài toán khác về tập điểm

ClosestPair.h

Description: Tìm cặp điểm có khoảng cách nhỏ nhất.

Time: $\mathcal{O}(n \log n)$

<cassert>, "Point.h" ac41a6, 18 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x)
      S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

Strings (9)

KMP.h

Description: pi[x] tính độ dài của tiền tố dài nhất của s kết thúc tại vị trí x, nhưng không được bằng chính s[0...x] (abacaba -> 0010123). Có thể dùng để tìm tất cả các vị trí xuất hiện của một chuỗi con. Time: O(n) Status: Đã kiểm chứng trên kattis:stringmatching

d4375c, 16 lines

```
vi pi(const string &s) {
  vi p(sz(s));
  rep(i, 1, sz(s)) {
    int g = p[i - 1];
    while (g && s[i] != s[g])
      g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vi match(const string &s, const string &pat) {
  vi p = pi(pat + '\0' + s), res;
```

```
  rep(i, sz(p) - sz(s), sz(p)) if (p[i] == sz(pat))
    res.push_back(i - 2 * sz(pat));
  return res;
}
```

Zfunc.h

Description: z[i] tính độ dài tiền tố chung dài nhất của hai chuỗi s[i:] và s, ngoại trừ z[0] = 0. Ví dụ: abacaba -> 0010301.
Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string &S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i, 1, sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

Description: Với mỗi vị trí trong chuỗi, tính: p[0][i] = bán kính palindrome chẵn dài nhất có tâm tại vị trí i. p[1][i] = bán kính palindrome lẻ dài nhất có tâm tại vị trí i (không tính ký tự ở chính giữa).
Time: $\mathcal{O}(N)$

e7ad79, 15 lines

```
array<vi, 2> manacher(const string &s) {
  int n = sz(s);
  array<vi, 2> p = {vi(n + 1), vi(n)};
  rep(z, 0, 2) for (int i = 0, l = 0, r = 0; i < n; i++) {
    int t = r - i + !z;
    if (i < r)
      p[z][i] = min(t, p[z][l + t]);
    int L = i - p[z][i], R = i + p[z][i] - !z;
    while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
      p[z][i]++, L--, R++;
    if (R > r)
      l = L, r = R;
  }
  return p;
}
```

MinRotation.h

Description: Tìm phép xoay vòng có thứ tự từ điển nhỏ nhất của một chuỗi.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

d07a42, 15 lines

```
int minRotation(string s) {
  int a = 0, N = sz(s);
  s += s;
  rep(b, 0, N) rep(k, 0, N) {
    if (a + k == b || s[a + k] < s[b + k]) {
      b += max(0, k - 1);
      break;
    }
    if (s[a + k] > s[b + k]) {
      a = b;
      break;
    }
  }
  return a;
}
```

Hashing.h

Description: Các hàm hỗ trợ băm chuỗi (String Hashing). Sử dụng modulo $2^{64} - 1$ thay vì 2^{64} để hạn chế va chạm trên các bộ test có chủ đích. Modulo này chậm hơn khoảng 2 lần và cần cài đặt phức tạp hơn. Ví dụ, với modulo 2^{64}, hai chuỗi Thue-Morse dạng ABBA... và BAAB... có thể cho cùng hash. Nếu dữ liệu đầu vào ngẫu nhiên, có thể thay bằng `typedef ull H;`. Ngoài ra, có thể dùng modulo $10^9 + 7$ nếu không cần xét đến Birthday Paradox.
Status: Đã kiểm thử bằng stress test

2d2a67, 46 lines

```
typedef uint64_t ull;
struct H {
  ull x;
  H(ull x = 0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) {
    auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64);
  }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11 + 3; // (order ~ 3e9; random also
    ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string &str) : ha(sz(str) + 1), pw(ha) {
    pw[0] = 1;
    rep(i, 0, sz(str)) ha[i + 1] = ha[i] * C + str[i],
                       pw[i + 1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string &str, int length) {
  if (sz(str) < length)
    return {};
  H h = 0, pw = 1;
  rep(i, 0, length) h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i, length, sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i - length]);
  }
  return ret;
}

H hashString(string &s) {
  H h{};
  for (char c : s)
    h = h * C + c;
  return h;
}
```

SuffixArray.h

Description: Xây dựng mảng hậu tố (suffix array) và mảng LCP của xâu s bằng thuật toán nhân đôi (prefix doubling) kết hợp counting sort. `sa` là thứ tự từ điển của các hậu tố: `sa[0] = sz(s)` (hậu tố rỗng, luôn đứng đầu), các hậu tố thật nằm ở `sa[1..n]`. `lcp[i]` là độ dài tiền tố chung dài nhất của hai hậu tố `sa[i-1]` và `sa[i]`, còn `lcp[0] = 0`. LCP của hai hậu tố bất kỳ bằng min của `lcp` trên một đoạn liên tục (dùng kèm RMQ.h). Số xâu con phân biệt của s bằng $n(n+1)/2$ trừ tổng `lcp`. Mọi ký tự phải nằm trong $[1, \texttt{lim})$ vì giá trị 0 dành cho ký tự kết thúc; mặc định `lim = 256` cho `string`. Với số nguyên lớn hơn, dùng `basic_string<int>` và đặt `lim` bằng giá trị lớn nhất cộng 1 (mọi giá trị phải ≥ 1).

Usage: `string s;`
`SuffixArray a(s); a.sa[i]; a.lcp[i]; long long distinct = (ll)n * (n + 1) / 2 - accumulate(all(a.lcp), 0LL);`
Time: $\mathcal{O}(N \log N)$

002257, 25 lines

```
struct SuffixArray {
  vi sa, lcp;
  template <class S> SuffixArray(const S &s, int lim = 256) {
    int n = sz(s) + 1, k = 0, a, b;
    vi x(begin(s), end(s) + 1), y(n), ws(max(n, lim)), rnk(n);
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i, 0, n) ws[x[i]]++;
      rep(i, 1, lim) ws[i] += ws[i - 1];
      for (int i = n; i--;)
        sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i, 1, n) a = sa[i - 1], b = sa[i],
             x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p
                 - 1
                                        : p++;
    }
    rep(i, 1, n) rnk[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rnk[i++]] = k)
      for (k &&k--, j = sa[rnk[i] - 1]; s[i + k] == s[j + k]; k
          ++)
        ;
  }
};
```

AhoCorasick.h

Description: Dùng để tìm kiếm nhiều mẫu (pattern) cùng lúc. Khởi tạo bằng AhoCorasick ac(patterns); trạng thái gốc của automaton có chỉ số 0. find(word) trả về tại mỗi vị trí ID của từ dài nhất kết thúc tại đó, hoặc -1 nếu không có từ nào kết thúc tại vị trí đó. findAll(-, word) tìm tất cả các từ (tối đa O(N * sqrt(N)) từ nếu không có pattern trùng nhau), nhóm theo vị trí bắt đầu và sắp xếp từ ngắn đến dài. Cho phép các pattern trùng nhau; không cho phép pattern rỗng. Muốn tìm từ dài nhất bắt đầu tại mỗi vị trí, hãy đảo ngược tất cả đầu vào. Với bảng chữ cái lớn, chia mỗi ký tự thành các nhóm bit và dùng bit đánh dấu để phân biệt ranh giới giữa các ký tự. Time: - Xây dựng: O(26N), với N là tổng độ dài các pattern. - find(x): O(N), với N là độ dài x. - findAll: O(NM).

<cassert> f35677, 71 lines

```
struct AhoCorasick {
  enum { alpha = 26, first = 'A' }; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string &s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int &m = N[n].next[c - first];
      if (m == -1) {
        n = m = sz(N);
        N.emplace_back(-1);
      } else
        n = m;
    }
    if (N[n].end == -1)
      N[n].start = j;
```

```
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string> &pat) : N(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1)
          ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start]) =
            N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string> &pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Thêm và xóa các đoạn khỏi một tập hợp các đoạn rời nhau. Khi thêm, đoạn mới sẽ được gộp với tất cả các đoạn giao hoặc tiếp giáp với nó. Các đoạn có dạng [đầu trái, đầu phải), tức là lấy đầu trái và không lấy đầu phải.
Time: $\mathcal{O}(logN)$

edce47, 28 lines

```
set<pii>::iterator addInterval(set<pii> &is, int L, int R) {
  if (L == R)
    return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L, R});
}

void removeInterval(set<pii> &is, int L, int R) {
  if (L == R)
    return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L)
    is.erase(it);
  else
    (int &)it->second = L;
  if (R != r2)
    is.emplace(R, r2);
}
```

IntervalCover.h

Description: Tìm tập chỉ số của số đoạn ít nhất cần chọn để phủ kín một đoạn cho trước. Các đoạn có dạng [đầu trái, đầu phải). Để hỗ trợ dạng [đầu trái, đầu phải], thay điều kiện (A) bằng '|| R.empty()'. Trả về tập rỗng nếu không thể phủ kín (hoặc nếu G rỗng).
Time: $\mathcal{O}(NlogN)$

9e9d8d, 19 lines

```
template <class T> vi cover(pair<T, T> G, vector<pair<T, T>> I)
    {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1)
      return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

ConstantIntervals.h

Description: Chia một hàm đơn điệu trên đoạn [from, to) thành tập các đoạn nửa kín nhỏ nhất sao cho hàm có giá trị không đổi trên mỗi đoạn. Gọi callback g cho từng đoạn như vậy.
Usage: `constantIntervals(0, sz(v), [](int x){return v[x];}, [](int lo, int hi, T val){...});`
Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

753a4c, 22 lines

```
template <class F, class G, class T>
void rec(int from, int to, F &f, G &g, int &i, T &p, T q) {
  if (p == q)
    return;
  if (from == to) {
    g(i, to, p);
    i = to;
    p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid + 1, to, f, g, i, p, q);
  }
}
template <class F, class G> void constantIntervals(int from,
     int to, F f, G g) {
  if (to <= from)
    return;
  int i = from;
  auto p = f(i), q = f(to - 1);
  rec(from, to - 1, f, g, i, p, q);
  g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Tìm chỉ số i nhỏ nhất trong [a, b] sao cho f(i) đạt giá trị lớn nhất, giả sử f(a) < ... < f(i) >= ... >= f(b). Để đảo ngược phía được phép có dấu bằng trong tính đơn điệu, đổi dấu < tại (A) thành <= và đảo ngược vòng lặp tại (B). Để tìm giá trị nhỏ nhất của f, đổi dấu < thành >, kể cả tại (B).
Usage: `int ind = ternSearch(0,n-1,[](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

<cassert> 9155b4, 12 lines

```
template <class F> int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid + 1))
      a = mid; // (A)
    else
      b = mid + 1;
  }
  rep(i, a + 1, b + 1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

LIS.h

Description: Tìm các chỉ số của dãy con tăng dài nhất (LIS).
Time: $\mathcal{O}(N \log N)$

2932a0, 20 lines

```
template <class I> vi lis(const vector<I> &S) {
  if (S.empty())
    return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i, 0, sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end())
      res.emplace_back(), it = res.end() - 1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it - 1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--)
    ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Cho trước N trọng số nguyên không âm w và một giới hạn không âm t, tìm giá trị lớn nhất S <= t sao cho S là tổng trọng số của một tập con các phần tử trong w.
Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 19 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t)
    a += w[b++];
  if (b == sz(w))
    return a;
  int m = *max_element(all(w));
  vi u, v(2 * m, -1);
  v[a + m - t] = b;
  rep(i, b, sz(w)) {
    u = v;
    rep(x, 0, m) v[x + w[i]] = max(v[x + w[i]], u[x]);
    for (x = 2 * m; --x > m;)
      rep(j, max(0, u[x]), v[x]) v[x - w[j]] = max(v[x - w[j]],
          j);
  }
  for (a = t; v[a + m - t] < 0; a--)
    ;
  return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: Khi thực hiện DP trên đoạn: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, nếu vị trí k tối ưu nhỏ nhất tăng theo cả i và j, ta có thể tính DP theo thứ tự độ dài đoạn tăng dần và chỉ cần tìm k = p[i][j] trong khoảng [p[i][j-1], p[i+1][j]]. Kỹ thuật này được gọi là Knuth DP. Điều kiện đủ để áp dụng là: $f(b,c) \le f(a,d)$ và $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$, với mọi $a \le b \le c \le d$. Ngoài ra, có thể tham khảo: LineContainer (chương Data structures), hàng đợi đơn điệu (Monotone Queue) và tìm kiếm tam phân (Ternary Search).
Time: $\mathcal{O}\left(N^2\right)$

DivideAndConquerDP.h

Description: Cho: $a[i] = \min_{lo(i)\le k<hi(i)}(f(i,k))$, trong đó chỉ số k tối ưu nhỏ nhất tăng theo i. Tính a[i] với i = L..R-1.
Time: $\mathcal{O}\left((N + (hi - lo))\log N\right)$

d38d2b, 19 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R)
      return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO, lo(mid)), min(HI, hi(mid))) best =
      min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second + 1);
    rec(mid + 1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

DigitDP.h

Description: Khung Digit DP để đếm các số trong đoạn [0, N] thỏa mãn một số điều kiện cho trước. Với đoạn [L, R], tính solve(R) - solve(L - 1).
Time: $\mathcal{O}\left(D * S * 10\right)$, D: số chữ số, S: số trạng thái.

cb5eda, 46 lines

```
struct DigitDP {
  int memo[20][200]; // Change base on problem
  vi dig;
  int dfs(int pos, int st, bool tight, bool lz) {
    if (pos == sz(dig)) {
      return (st == 0); // Base case
    }
    if (!tight && !lz && memo[pos][st] != -1) {
      return memo[pos][st];
    }
    int lim = tight ? dig[pos] : 9;
    int res = 0;
    for (int d = 0; d <= lim; ++d) {
      bool nlz = lz && (d == 0);
      int nst =
          transition(st, d, nlz); // Change status function
      res += dfs(pos + 1, nst, tight && (d == lim), nlz);
    }
    if (!tight && !lz) {
      memo[pos][st] = res;
    }
    return res;
  }
  int transition(int st, int d, bool nlz) {
    // Customize the state transition formula according to the
    // problem. For example: calculate the digit sum, count
    // occurrences of a specific digit, etc.
    if (nlz)
      return st; // If still processing leading zeros, keep
          // the state unchanged.
    return st + d;
  }
  int solve(long long n) {
    if (n < 0)
      return 0;
    if (n == 0)
      return 1; // Adjust depending on whether zero should be
          // counted as valid.
    dig.clear();
    for (; n > 0; n /= 10)
      dig.push_back(n % 10);
    reverse(all(dig));
    memset(memo, -1, sizeof(memo));
    return dfs(0, 0, true, true);
  }
};
```

- `signal(SIGSEGV, [](int) { _Exit(0); });` biến segfault thành Wrong Answer. Tương tự, có thể bắt SIGABRT (assertion thất bại) và SIGFPE (chia cho không). Lỗi của `_GLIBCXX_DEBUG` sinh ra SIGABRT (hoặc SIGSEGV trên gcc 5.4.0).
- `feenableexcept(29);` dừng chương trình khi gặp NaN (1), chia cho 0 (4), vô cực (8) và số không chuẩn hóa (denormal) (16).

10.4 Mẹo tối ưu

`__builtin_ia32_ldmxcsr(40896);` tắt denormal (denormal làm số thực chậm hơn 20 lần khi gần giá trị nhỏ nhất).

10.4.1 Thủ thuật bit

- `x & -x` là bit thấp nhất đang bật của `x`.
- `for (int x = m; x; ) { --x &= m; ... }` duyệt qua mọi mặt nạ con của `m` (trừ chính `m`).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` là số kế tiếp sau `x` có cùng số bit được bật.
- `rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)];` tính tổng trên mọi tập con.

10.4.2 Pragma

- `#pragma GCC optimize ("Ofast")` giúp GCC tự động vector hóa vòng lặp và tối ưu số thực tốt hơn.
- `#pragma GCC target ("avx2")` có thể tăng gấp đôi hiệu năng của code đã vector hóa, nhưng gây crash trên máy cũ.
- `#pragma GCC optimize ("trapv")` dừng chương trình khi tràn số nguyên (nhưng rất chậm).

FastMod.h

Description: Tính $a\%b$ nhanh hơn khoảng 5 lần so với cách thông thường, với b là hằng số nhưng không biết tại thời điểm biên dịch. Trả về một giá trị đồng dư với $a \pmod b$ trong khoảng $[0, 2b)$.

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;
  }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.

7b3c70, 17 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

BumpAllocator.h

Description: Dùng khi cần cấp phát động nhiều đối tượng và không cần giải phóng chúng. "new X" thông thường có chi phí phụ khoảng 0.05us + 16 byte cho mỗi lần cấp phát.

<cassert> 745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void *operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
```

```
  return (void *)&buf[i -= s];
}
void operator delete(void *) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind); }
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

Usage: `vector<vector<int, small<int>>> ed(N);` bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
  typedef T value_type;
  small() {}
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T);
    return (T*)(buf + buf_ind);
  }
  void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern "`_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)`". Not all are described here; grep for `_mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and #define `__SSE__` and `__MMX__` before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer loadu/storeu.

c9ac08, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
  int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
  int i = 0; ll r = 0;
  mi zero = _mm256_setzero_si256(), acc = zero;
  while (i + 16 <= n) {
    mi va = L(a[i]), vb = L(b[i]); i += 16;
    va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
    mi vp = _mm256_madd_epi16(va, vb);
    acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
  }
  union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
  for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
  return r;
}
```